



POZNAN UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTING AND TELECOMMUNICATION
Institute of Computing Science

Master's thesis

COMPARATIVE STUDY OF COMPATIBLE SUBSTITUTIONS OPTIMIZATION AND LATENT SPACE OPTIMIZATION IN EVOLUTIONARY DESIGN

Filip Rosiak, 151799
Jakub Stefański, 151876

Supervisor
dr hab. inż. Maciej Komosiński

POZNAŃ 2026

Tutaj będzie karta pracy dyplomowej;
oryginał wstawiamy do wersji dla archiwum PP, w pozostałych kopiach wstawiamy ksero.

Abstract

This thesis investigates optimization methods for variable-length, non-positional genotypes in evolutionary design. Framsticks **f1** genotypes are used as the main case study because their tree-like structure makes local genotype modifications strongly context-dependent. In such representations, the effect of a fragment depends on its surrounding structure, which makes standard mutation and crossover operators prone to disrupting useful substructures.

The work compares two complementary approaches to this problem. The first one, Compatible Substitutions Optimization (CoSO), operates directly in the discrete genotype representation and attempts to guide substitutions by estimating their contextual compatibility. The second one, Latent Space Optimization (LSO), maps genotypes into a continuous latent representation learned by autoencoders and performs search in that space. The autoencoder-based part includes grammar-aware models such as TreeVAE and TransformerVAE, as well as variants using grammatical context, depth information, and fitness conditioning. The optimized candidates are decoded back into **f1** genotypes and evaluated with the Framsticks fitness criterion *vertpos*.

The experimental evaluation is designed to compare CoSO, latent-space optimization methods, and standard Framsticks evolutionary operators under a common protocol. The comparison considers the quality of the best solutions found, improvement over the initial genotypes, stability across starting points, and the influence of the initial fitness range. The results are intended to show when explicit context-aware substitutions and learned continuous representations are useful, what their limitations are, and how the choice of representation affects the optimization process.

Streszczenie

Niniejsza praca dotyczy metod optymalizacji genotypów o zmiennej długości i niepozycyjnym charakterze reprezentacji w projektowaniu ewolucyjnym. Jako główny przypadek badawczy wykorzystano genotypy Framsticks **f1**, których drzewiasta struktura sprawia, że lokalne modyfikacje genotypu są silnie zależne od kontekstu. W takich reprezentacjach znaczenie fragmentu zależy od otaczającej go struktury, przez co standardowe operatory mutacji i krzyżowania mogą niszczyć użyteczne podstruktury.

W pracy porównywane są dwa uzupełniające się podejścia do tego problemu. Pierwsze z nich, Compatible Substitutions Optimization (CoSO), działa bezpośrednio w dyskretnej reprezentacji genotypu i wykorzystuje ocenę zgodności kontekstowej proponowanych podmian. Drugie, Latent Space Optimization (LSO), odwzorowuje genotypy do ciągłej przestrzeni ukrytej wyuczonej przez autoenkodery i prowadzi optymalizację w tej przestrzeni. Część autoenkodera obejmuje modele świadome gramatyki, takie jak TreeVAE i TransformerVAE, a także warianty wykorzystujące kontekst gramatyczny, informację o głębokości oraz warunkowanie wartością fitness. Kandydaci wygenerowani w przestrzeni ukrytej są dekodowani z powrotem do genotypów **f1** i oceniani przy użyciu kryterium *vertpos* w środowisku Framsticks.

Ewaluacja eksperymentalna została zaprojektowana tak, aby porównać CoSO, metody optymalizacji w przestrzeni ukrytej oraz standardowe operatory ewolucyjne Framsticks we wspólnym protokole. Porównanie obejmuje jakość najlepszych znalezionych rozwiązań, poprawę względem genotypów startowych, stabilność między różnymi punktami startowymi oraz wpływ początkowego zakresu fitness. Celem analizy jest określenie, kiedy jawne podmiany zgodne z kontekstem oraz wyuczone reprezentacje ciągłe są użyteczne, jakie są ich ograniczenia oraz w jaki sposób wybór reprezentacji wpływa na proces optymalizacji.

Contents

Abstract	I
Streszczenie	II
1 Introduction	1
1.1 Problem statement	1
1.2 Motivation	2
1.3 Aim and scope of the thesis	3
1.4 Proposed research approach	3
1.5 Thesis structure	4
2 Literature and Background	5
2.1 Evolutionary design and variable-length genotypes	5
2.2 Framsticks and the f1 genetic representation	6
2.3 Context-aware genotype modification	7
2.4 Compatible Substitutions Optimization	8
2.5 Autoencoders and variational autoencoders	9
2.6 Latent Space Optimization	10
3 Methods and Implemented Components	12
3.1 Dataset and fitness evaluation	12
3.2 Grammar-based representation of f1 genotypes	13
3.3 Tree-based fragment handling	14
3.4 Compatible Substitutions Optimization module	15
3.5 Autoencoder-based genotype representation	16
3.6 Latent-space optimization methods	18
3.7 Baseline evolutionary methods	20
3.8 Implementation architecture	21
4 Experimental Design	23
4.1 Research questions	23
4.2 Compared methods	23
4.3 Benchmark setup	25
4.4 Evaluation metrics	26
4.5 Experimental protocol	27
4.6 Reproducibility and computational setup	30
5 Experimental Results	31
5.1 Autoencoder reconstruction and validity results	31

5.2	CoSO optimization results	32
5.3	Latent-space optimization results	34
5.4	Comparison with the Framsticks evolutionary baseline	35
5.5	Comparison between CoSO and Latent Space Optimization	36
5.6	Analysis by initial fitness ranges	38
5.7	Summary of results	40
6	Discussion and Conclusions	42
6.1	Strengths and limitations of Compatible Substitutions Optimization	42
6.2	Strengths and limitations of Latent Space Optimization	43
6.3	Comparative discussion	45
6.4	Threats to validity	45
6.5	Conclusions	45
6.6	Future work	45
	Bibliography	46
A	Supplementary Materials	48
A.1	Additional experimental results	48
A.2	Configuration details	48
A.3	Examples of optimized genotypes	48

Chapter 1

Introduction

1.1 Problem statement

Evolutionary design studies the automatic generation and improvement of artefacts by means of evolutionary computation. Instead of manually specifying a complete design, a human designer defines a representation, variation operators, and a fitness criterion, while the search process explores possible artefacts. Such artefacts may include engineering structures, robot morphologies, artificial organisms, programs, or other objects whose quality can be assessed by an objective function or by simulation. This makes evolutionary design a broad research area rather than a technique tied to one particular environment or representation [1].

A central issue in evolutionary design is the choice of representation. The representation determines which candidate designs can be expressed, how variation changes them, and how genotype-level modifications are translated into phenotype-level effects. Previous work distinguishes many representation families, including direct and indirect encodings, fixed-length and variable-length genotypes, linear and non-linear structures, grammatical encodings, developmental encodings, graph-based representations, and generative representations [1, 10]. These alternatives differ not only in expressiveness, but also in the structure of the search space that an optimization algorithm has to explore.

The problem considered in this thesis belongs to the class of optimization problems in which candidate solutions are represented by structured, variable-length, non-positional genotypes. In such representations, the meaning of a fragment is not determined only by its local textual form or by a fixed coordinate. Instead, its effect may depend on syntax, nesting and neighbouring fragments. A local modification that is meaningful in one context may be invalid, neutral, or harmful in another. This context dependence makes the definition of useful variation operators and neighbourhoods non-trivial.

The thesis compares two representation-aware approaches to this problem. Compatible Substitutions Optimization (CoSO) operates directly in the discrete genotype space and attempts to propose substitutions between fragments that are compatible with their surrounding context. Latent Space Optimization (LSO) follows a different strategy: it learns a continuous representation of genotypes and performs search in that learned space before decoding candidates back to the original representation. CoSO can therefore be interpreted as an explicit, context-aware discrete approach, whereas LSO is an implicit, learned, continuous-representation approach.

The empirical part of the thesis instantiates this general problem using Framsticks **f1** genotypes as a case study. **f1** genotypes are variable-length symbolic descriptions with a branching, tree-like interpretation, and candidate solutions are evaluated in Framsticks with respect to the *vertpos*

criterion. Framsticks is therefore used as a concrete experimental environment for comparing CoSO, LSO, and a standard evolutionary baseline, rather than as the only possible domain to which the studied approaches apply.

1.2 Motivation

Evolutionary design is motivated by the possibility of exploring large and complex design spaces automatically. In many domains, useful artifacts are difficult to design by explicitly specifying every component and relation in advance. Evolutionary methods make it possible to search over alternative structures, morphologies, or construction rules, provided that candidate artifacts can be represented, varied, and evaluated. This is especially useful when the quality of a design emerges only after simulation or phenotype construction.

The motivation for this thesis comes from the observation that representation strongly influences what an evolutionary search can discover. A simple fixed-length vector can be convenient for optimization, but it may be poorly suited for designs whose size, topology, or hierarchy should change during search. More expressive representations, including developmental and generative encodings, can support modularity, reuse, hierarchy, and structural regularity, but they also make variation more difficult to control [2, 8, 10]. Therefore, improving search in evolutionary design often requires methods that are aware of the representation rather than operators that treat all positions or symbols independently.

Variable-length structured representations highlight this challenge particularly clearly. Candidate solutions are not simple collections of independent parameters. They may contain fragments whose interpretation depends on grammar, position in a tree, branch structure, or the construction process that maps genotype to phenotype. Random local edits may explore the space widely, but they can also destroy syntactic validity or disrupt useful substructures. This creates practical motivation to study optimization methods that preserve useful fragments and propose modifications in a more context-sensitive way.

Compatible Substitutions Optimization (CoSO) is motivated by the need for explicit context awareness in the original genotype space. Instead of treating a substitution as an isolated textual edit, CoSO considers whether the inserted fragment is compatible with the recipient context. This keeps the method close to the discrete representation and makes the proposed modifications interpretable as concrete genotype-level changes. However, compatibility does not by itself guarantee improvement, so candidate substitutions must still be verified by the true fitness evaluation.

Latent Space Optimization (LSO) is motivated by a complementary idea. Rather than manually defining all useful neighbourhood relations in the symbolic genotype space, an autoencoder may learn a continuous representation in which related genotypes are mapped to nearby latent vectors. If this representation captures relevant syntactic or structural regularities, continuous search methods can be applied in the latent space and the resulting candidates can be decoded back into genotypes. This approach may smooth the search space, but it also introduces risks related to reconstruction quality, validity, diversity, and the relation between latent-space movement and actual fitness improvement.

The comparison between CoSO and LSO is therefore motivated by a broader question: how should optimization methods handle context-dependent, variable-length representations in evolutionary design? CoSO represents an explicit and discrete way of using representation knowledge, while LSO represents a learned and continuous way of constructing a search space. Framsticks **f1** and the *vertpos* objective are used in this thesis as a concrete experimental case study for this comparison.

1.3 Aim and scope of the thesis

The main aim of this thesis is to compare two representation-aware approaches to optimization in evolutionary design: Compatible Substitutions Optimization (CoSO) and Latent Space Optimization (LSO). Both approaches address the broader difficulty of optimizing structured, variable-length, context-dependent genotypes. The comparison is not intended to prove that either approach universally replaces standard evolutionary algorithms. Instead, the thesis aims to determine how explicit context-aware substitutions and learned continuous representations behave when they are applied to the same class of structured genotypes and evaluated under a common objective.

The empirical scope of the work instantiates this general problem using Framsticks **f1** genotypes optimized for the *vertpos* fitness criterion. This restriction makes it possible to define a controlled case study in which all candidate solutions are ultimately evaluated by the same simulation-based criterion. Within this scope, CoSO represents the discrete and explicit direction of the study, while LSO represents the learned continuous-representation direction.

The CoSO part of the thesis covers direct genotype-level optimization through tree-based fragment handling, compatibility-based fragment matching, archive-supported substitutions, genotype repair, and true Framsticks evaluation of generated candidates. The focus is on whether representation-aware substitutions can preserve useful structural fragments while still allowing the search to improve candidate solutions.

The LSO part of the thesis covers preparing **f1** genotype data for autoencoder training, representing genotypes with character-based or grammar-aware encodings, training selected autoencoder architectures, decoding latent vectors back into **f1** genotypes, and applying optimization methods in the learned latent space. The autoencoder-based investigation focuses on whether the learned representation can support productive search, not only on whether it reconstructs training examples accurately.

The thesis also includes a baseline comparison with Framsticks EA. This baseline is important because it represents the native evolutionary search procedure for the considered environment and provides a practical reference point for both CoSO and LSO. The intended outcome of the study is therefore a comparative assessment of solution quality, validity, stability, and dependence on starting conditions, rather than a single claim that one method is always superior.

1.4 Proposed research approach

The research approach follows a comparison of two representation-aware optimization strategies under a common evaluation protocol. To make this comparison concrete and experimentally controlled, the thesis uses Framsticks **f1** genotypes and the *vertpos* objective as the shared case study. This provides a common basis for comparing methods that operate in different search spaces while preserving the same final phenotype-level evaluation.

For CoSO, the thesis considers optimization through context-aware substitutions in the original discrete genotype representation. The method extracts coherent genotype fragments, compares them using representation-level compatibility descriptors, and tests substitutions in complete candidate genotypes. This approach is analysed as an explicit attempt to use information about the compatibility of genotype fragments with their surrounding context.

For LSO, the research approach consists of learning continuous representations of **f1** genotypes with autoencoders and then applying search operators to vectors in the latent space. A candidate latent vector is decoded into an **f1** genotype and evaluated with the same Framsticks

fitness criterion as the baseline. This makes it possible to compare latent-space search with direct genotype-level optimization while preserving a common final evaluation procedure. The implemented LSO component considers several families of autoencoder models and several continuous or population-based optimization strategies, but their technical details are introduced only in the methods and experimental chapters.

The comparison is designed to separate three related questions. First, it examines whether generated or modified genotypes remain valid and evaluable. Second, it assesses whether the search process improves the fitness of starting genotypes. Third, it compares the stability of methods across different starting conditions. This structure reflects the fact that a representation can be syntactically convenient without necessarily producing better optimization results, and that a high-performing single run is not sufficient evidence of a stable optimization method.

1.5 Thesis structure

The remainder of the thesis is organized as follows. Chapter 2 presents the literature and background required for the study. It introduces evolutionary design with variable-length genotypes, the Framsticks environment and the **f1** representation, context-aware genotype modification, Compatible Substitutions Optimization, autoencoders, variational autoencoders, and the general idea of Latent Space Optimization.

Chapter 3 describes the methods and implemented components. It covers the dataset and fitness evaluation procedure, the grammar-based handling of **f1** genotypes, the CoSO component, the autoencoder representations, latent-space optimization methods, baseline evolutionary methods, and the software architecture used to run the experiments.

Chapter 4 defines the experimental design. It formulates the research questions, identifies the compared methods, describes the benchmark setup and evaluation metrics, and explains the experimental protocol and reproducibility considerations. Chapter 5 reports the experimental results, including reconstruction and validity analyses for autoencoders, CoSO results, latent-space optimization results, comparisons with Framsticks EA, a direct comparison between CoSO and Latent Space Optimization, and analyses conditioned on the initial fitness range.

Chapter 6 discusses the results and concludes the thesis. It summarizes the main findings, analyses the strengths and limitations of CoSO and LSO, compares the two approaches, discusses threats to validity, and outlines possible directions for future work. Supplementary material is provided in the appendix where appropriate.

Chapter 2

Literature and Background

2.1 Evolutionary design and variable-length genotypes

Evolutionary design is the use of computational search to automatically generate or improve artefacts whose quality can be assessed by an objective function. In contrast to conventional optimization problems with a small number of predefined numerical parameters, evolutionary design often concerns structured objects: shapes, mechanisms, controllers, artificial organisms, or other constructions whose form is not known in advance. The search process therefore has to explore both parameter values and structural alternatives. Candidate solutions are usually represented as genotypes, transformed into phenotypes, and evaluated according to a task-dependent fitness criterion. Selection and variation operators then bias the population toward solutions with better observed performance.

A distinctive feature of many evolutionary design problems is that the genotype does not have a fixed and directly positional interpretation. In a fixed-length vector representation, the meaning of a coordinate is typically stable across all individuals: the same index denotes the same parameter. By contrast, a variable-length genotype can contain fragments whose role depends on their position, nesting, and surrounding symbols. Such genotypes may encode branching morphology, repeated substructures, hierarchical components, or program-like construction rules. Their length may change during mutation and recombination, and a local edit can alter not only one parameter but also the interpretation of a larger downstream structure.

This property makes evolutionary design more expressive, but it also makes optimization more difficult. The search space becomes discrete, irregular, and strongly constrained by syntactic and semantic validity. Some candidate strings may not describe any admissible phenotype, while others may be valid but differ greatly in behaviour after only a small textual change. Furthermore, the contribution of a genotype fragment is often context-dependent: a fragment that is useful in one structural location can be neutral, invalid, or harmful in another. This context dependence is closely related to epistasis, understood broadly as interaction between parts of the representation, where the effect of modifying one part depends on other parts of the genotype.

Simulation-based optimization adds another layer of complexity. In this setting, fitness is not obtained from a closed-form mathematical expression, but from a simulation that constructs and evaluates the phenotype. This is typical for artificial-life and evolutionary robotics environments, where the performance of a morphology may depend on physical interaction, dynamics, control, or environmental conditions. Simulation enables the assessment of complex behaviours, but it also makes each evaluation relatively expensive and sometimes noisy. As a result, variation operators should ideally produce candidates that are not only diverse, but also valid and promising enough

to justify evaluation.

The representation used in this thesis belongs to this class of problems. Framsticks **f1** genotypes are variable-length symbolic descriptions of virtual organisms, and their fitness is obtained after phenotype construction and simulation. The *vertpos* criterion considered later in the thesis is therefore an evaluated property of the resulting organism, not a simple property of the raw character string. This distinction is important because a method that performs well at modifying genotype text does not necessarily perform well at improving simulated fitness. Conversely, an optimization method may be valuable if it discovers modifications that preserve validity and produce useful phenotypic effects, even when the mapping from genotype to fitness is highly indirect.

The background developed in this section motivates the two representation-aware directions studied in the thesis. Compatible Substitutions Optimization (CoSO) addresses the difficulty directly in the discrete genotype representation by considering compatibility between a substitution and its context. Latent Space Optimization (LSO) approaches the same difficulty by learning a continuous representation in which variable-length genotypes are mapped to fixed-dimensional vectors before search is performed. Both directions can be understood as attempts to define more useful neighbourhoods for structured genotypes than those obtained from naive position-independent edits. The following sections introduce the concrete Framsticks **f1** representation and then discuss these two approaches in greater detail.

2.2 Framsticks and the **f1** genetic representation

Framsticks is an artificial-life and evolutionary-design environment in which virtual organisms are described by genotypes, developed into phenotypes, simulated, and evaluated according to selected criteria. For the purposes of this thesis, Framsticks provides both the design domain and the final evaluator of candidate solutions. The methods compared later operate on, or generate, symbolic genotype descriptions, but their usefulness is ultimately determined by whether these descriptions can be interpreted as valid organisms and assessed in simulation. The environment therefore connects representation-level search with phenotype-level evaluation.

The representation considered in this thesis is Framsticks **f1**. At a conceptual level, an **f1** genotype is a textual description of an organism rather than a fixed list of numerical parameters. Its symbols specify structural components and modifiers, and the complete string is interpreted through a genotype-to-phenotype mapping. This mapping is indirect: the fitness of a candidate is not defined by the string alone, but by the organism produced from it and by the behaviour of this organism during simulation. The later use of the *vertpos* criterion should therefore be understood as evaluation of a constructed phenotype, not as a direct score assigned to individual characters.

f1 genotypes are variable-length expressions. This property is important because evolutionary variation can change not only parameter values, but also the amount and organization of encoded structure. Longer or shorter genotypes may describe organisms with different numbers of components, different branching patterns, or different modifier sequences. Consequently, the search space is not a simple Euclidean space with a fixed dimensionality. It is a discrete structured space in which insertion, deletion, and substitution can change the interpretation of neighbouring fragments and, in some cases, larger parts of the organism.

The **f1** representation also contains branching and nested descriptions. In simplified grammatical terms, genotype strings may be understood as sequences of elements, where elements can contain a core component, optional modifiers, and branch expressions enclosed in parentheses. Branches make the representation expressive, because they allow the genotype to describe articulated or tree-like morphology. At the same time, they impose syntactic constraints: parentheses,

separators, branch contents, and modifier positions must be arranged in forms that can be parsed and developed into a phenotype. A candidate that violates these constraints may be useless for simulation-based optimization because it cannot be evaluated in the intended way.

Syntactic validity is therefore a central issue for both approaches studied in the thesis. In a direct genotype-level method, a local edit must preserve enough structure for the resulting expression to remain acceptable. In an autoencoder-based method, a decoder must produce strings that are not merely character sequences, but valid **fl** genotypes. Validity is not equivalent to good performance, but invalidity prevents meaningful phenotype evaluation. This distinction motivates the later separation between reconstruction or validity analysis and actual optimization performance.

Another important property of **fl** is context sensitivity. The effect of a symbol or fragment depends on where it appears in the genotype and on the structural role of the surrounding expression. A modifier before a core component, a symbol inside a branch, and a separator between branches are not interchangeable even if they are all local textual tokens. Therefore, edits that ignore context may easily produce invalid genotypes or changes whose phenotypic consequences are difficult to control. This observation prepares the ground for context-aware genotype modification and for Compatible Substitutions Optimization, where the compatibility of a substitution with its surrounding expression is treated as an essential concern.

The same representational properties also motivate Latent Space Optimization. Since **fl** genotypes are variable-length, branching, and syntactically constrained, mapping them to fixed-dimensional continuous vectors is non-trivial. A useful latent representation should preserve enough information about the genotype structure to allow decoding back into valid candidates, while also arranging genotypes in a way that supports search. The background role of the **fl** representation is thus twofold: it defines the concrete optimization domain of the thesis, and it explains why both explicit context-aware edits and learned continuous representations are relevant alternatives to naive character-level modification.

2.3 Context-aware genotype modification

Context-aware genotype modification is the design of variation operators that take into account the role of the edited fragment within the whole genotype. In a simple positional representation, a mutation can often be described as changing the value of one coordinate, and a recombination operator can exchange values at selected indices. In a structured variable-length representation, this view is insufficient. A fragment may be useful only because it appears after a particular prefix, inside a particular branch, or in a location where its syntactic and phenotypic consequences fit the surrounding construction. A context-aware operator therefore attempts to modify a genotype in units that are meaningful for the representation and to avoid edits that unnecessarily disrupt other parts of the solution.

This idea is closely related to the concept of linkage in evolutionary computation. Linkage describes dependencies between variables or fragments that should be considered jointly during variation. If strongly dependent elements are modified independently, useful partial structures may be destroyed; if unrelated elements are always modified together, the search may lose flexibility. Linkage-aware algorithms therefore try to identify groups of variables that form useful building blocks and to process these groups as units. Bosman and Thierens describe this principle in the context of optimal mixing evolutionary algorithms, where a linkage model is represented as a Family of Subsets (FOS), and Gene-pool Optimal Mixing (GOM) uses this model to copy selected groups of variables from donor solutions into a recipient solution [4]. The important point for this

thesis is the way in which it evaluates partial modifications after they have been inserted into a complete solution. The usefulness of a copied fragment is therefore judged in context, by its effect on the modified recipient, rather than as an isolated property of the fragment itself.

The direct use of such positional linkage models is natural for fixed-length genotypes, but it becomes difficult for variable-length and non-positional representations. In these representations, the same functional or structural fragment does not have to appear at the same textual index in different individuals. A subtree, branch, modifier sequence, or construction fragment may be transferable even when it occurs at a different position, while two fragments at the same approximate position may have incompatible roles. Consequently, context-aware modification for variable-length genotypes cannot rely only on fixed coordinate sets. It must instead define what it means for two fragments to be compatible in their local surroundings.

For genotype representations such as Framsticks **f1**, compatibility has both syntactic and phenotypic aspects. A replacement fragment should preserve enough grammatical structure for the resulting genotype to remain valid, but validity alone is not sufficient. The substitution may also change how subsequent parts of the genotype are developed into the phenotype. A useful compatibility criterion should therefore estimate whether replacing one fragment with another is likely to preserve the relationship between the unchanged prefix, the inserted fragment, and the unchanged suffix or surrounding branch structure. Such a criterion does not need to predict the final fitness precisely; rather, it defines a neighbourhood in which local changes are less likely to destroy the contribution of the rest of the genotype. This observation leads directly to Compatible Substitutions Optimization, where the neighbourhood of a genotype is defined by substitutions between fragments that are considered compatible before they are tested by the true fitness evaluator.

2.4 Compatible Substitutions Optimization

Compatible Substitutions Optimization (CoSO) applies the above idea by replacing genotype fragments only when they are considered suitable for the context in which they are inserted. The method can be viewed as a variable-length, non-positional counterpart of optimal mixing. It preserves the intuition that useful building blocks should be reused across solutions, but it does not require these building blocks to occupy the same fixed indices in every genotype. Instead of asking which positions should be copied together, CoSO asks which sub-solution can replace another sub-solution without strongly disturbing the remaining genotype [12].

The central concept of CoSO is the Substitution Compatibility Function (SCF). This function compares two sub-solutions: a fragment currently present in a recipient genotype and a candidate fragment obtained from a donor genotype or from a collection of previously observed fragments. A low compatibility value means that the donor fragment is expected to be a relatively safe replacement for the recipient fragment. The function may express representation-specific knowledge about syntax, geometry, construction rules, or the role played by the fragment in the phenotype. For this reason, CoSO is a gray-box approach: it does not require a full predictive model of fitness, but it assumes that some useful knowledge about the representation can be used to restrict the set of substitutions worth evaluating.

The general optimization step follows a simple pattern. Candidate sub-solutions are extracted from available genotypes, and potential substitutions are formed by matching recipient fragments with donor fragments whose compatibility score is below an accepted threshold. Each proposed substitution is then applied to the complete recipient genotype and evaluated as part of that complete solution. As in optimal mixing, the value of a fragment is therefore not assessed in isolation. A substitution is useful only if the modified genotype improves the evaluated objective,

and non-improving substitutions are rejected. This creates a search process that is local with respect to compatible fragment replacements, while still using the true fitness function as the final criterion.

This formulation is especially relevant for variable-length genotypes. A donor fragment may be useful even if it appears at a different position than the fragment it replaces, provided that it fits the local structural context of the recipient. Conversely, two fragments with similar textual positions may be poor substitutes if their structural roles differ. CoSO therefore separates the notion of a reusable building block from fixed genotype coordinates. It treats sub-solutions as transferable objects whose usefulness depends on compatibility with the recipient context.

In the **f1** setting considered in this thesis, this perspective motivates comparing fragments through descriptors of their structural effect rather than through raw textual positions alone. A fragment may be characterized by properties related to how it changes the constructed structure, and fragments with similar effects are more natural candidates for substitution. Such descriptors do not guarantee that a replacement will improve *vertpos*; they only make the proposed modification less arbitrary. CoSO should therefore be understood as a compromise between blind random editing and full semantic prediction: it uses representation knowledge to propose less disruptive candidates, but it still relies on empirical Framsticks evaluation to determine whether a proposed modification is actually beneficial.

2.5 Autoencoders and variational autoencoders

Autoencoders are representation-learning models trained to reproduce their inputs after passing them through an intermediate code. The model is usually divided into an encoder and a decoder. The encoder maps an input object to a latent representation, while the decoder attempts to reconstruct the original object from this representation. If the latent representation is constrained to be lower-dimensional, regularized, or otherwise information-limited, the model is encouraged to capture regularities that are useful for reconstruction rather than simply copying the input [17, 7].

This encoder-decoder structure is particularly relevant for the study of structured genotypes. A genotype such as a Framsticks **f1** string is variable-length and symbolic, whereas many optimization methods operate most naturally on fixed-size numerical vectors. An autoencoder can therefore be used as a bridge between these two forms. The encoder converts a genotype into a vector in the latent space, and the decoder converts a latent vector back into a candidate genotype. In this thesis, this idea is important not because reconstruction is an end in itself, but because reconstruction defines whether the learned representation can be used as a search space for genotype generation and modification.

Variational autoencoders extend this idea by making the latent representation probabilistic. Instead of encoding an input as a single unconstrained vector only, a variational autoencoder represents the input by parameters of a latent distribution, typically mean and variance parameters, and samples a latent vector from this distribution during training. The training objective combines reconstruction accuracy with a regularization term that encourages the latent distribution to remain close to a simple prior, often a standard normal distribution. This regularization is intended to make the latent space more continuous and more suitable for sampling, interpolation, and optimization than an arbitrary collection of isolated codes [11, 15].

For continuous data, reconstruction can often be measured by numerical distance. For discrete structured objects, however, reconstruction quality has additional dimensions. A decoded sequence may differ from the input by only a small number of symbols and still be syntactically invalid. Conversely, a reconstruction that is textually different from the input may still describe a valid

and potentially useful organism. Therefore, when autoencoders are applied to **f1** genotypes, the central concerns are not limited to token-level accuracy. The decoded output must also respect the constraints of the representation so that it can be interpreted as a valid Framsticks genotype and evaluated with a criterion such as *vertpos*.

This validity issue explains why grammar- and structure-aware decoders are relevant. A purely character-level decoder treats generation mainly as a sequence prediction task and must learn syntactic constraints indirectly from examples. This may be sufficient for simple patterns, but it provides no explicit guarantee that parentheses, branches, modifiers, and core elements will appear in admissible combinations. By contrast, grammar-aware representations can decode sequences of production rules rather than raw characters, and masked variants can restrict choices to rules that are valid for the currently expanded nonterminal [13]. Tree-oriented variants can additionally reflect the fact that **f1** genotypes contain branching structure rather than only a flat string. Such mechanisms do not by themselves guarantee high fitness, but they can reduce the burden placed on the model by embedding part of the known syntax into the decoder.

In the context of this thesis, autoencoders and variational autoencoders therefore provide the background for the Latent Space Optimization part of the comparison. They define a learned continuous representation of **f1** genotypes that is different from the explicit discrete representation used by context-aware substitutions and CoSO. The usefulness of this representation depends on several separate properties: whether inputs can be encoded and reconstructed, whether decoded samples remain valid genotypes, whether nearby latent points correspond to related genotype structures, and whether movement in the latent space is correlated with changes in simulation-based fitness. These properties are connected, but none of them alone is sufficient to establish that an autoencoder is useful for optimization.

2.6 Latent Space Optimization

Latent Space Optimization (LSO) is the use of a learned latent representation as the domain in which search is performed. Instead of applying variation directly to the original object, an object is encoded into a continuous vector, optimization modifies or samples vectors in this latent space, and each candidate vector is decoded back into the original representation before evaluation. Related work has used learned continuous representations for molecular design, program synthesis, and evolutionary search in learned representations, illustrating the general idea that an autoencoder or another generative model can define a search space for structured objects [5, 9, 14, 3]. In the setting considered in this thesis, the original objects are Framsticks **f1** genotypes, and the final objective remains the true simulation-based *vertpos* fitness. The latent vector is therefore an intermediate representation for proposing candidates, not a replacement for evaluation in Framsticks.

The motivation for LSO is that a learned continuous representation may provide a more compact or smoother search domain than the raw genotype space. **f1** genotypes are variable-length symbolic expressions with branching syntax and context-dependent effects. Direct edits in such a space can easily produce invalid candidates or large unpredictable phenotypic changes. If an autoencoder has learned useful regularities of the genotype distribution, nearby latent vectors may decode to related genotypes, and continuous perturbations may correspond to moderate structural changes. This can make it possible to apply optimization techniques that are difficult to define directly on raw **f1** strings.

The general LSO loop consists of four conceptual steps. First, one or more starting genotypes are encoded into latent vectors. Second, an optimizer proposes new latent vectors by mutation, recombination, distribution update, or another continuous-space operation. Third, each proposed

vector is decoded into an **f1** genotype. Fourth, the decoded genotype is checked for validity and evaluated in Framsticks, where its *vertpos* value provides feedback to the optimizer. This loop preserves a common final evaluation criterion with direct evolutionary search: regardless of how a candidate is generated, it is useful only if it decodes into a genotype that can be evaluated and achieves good simulated performance.

Several optimizer families can be used within this framework. An evolutionary search in latent space keeps a population of latent vectors, selects better candidates according to decoded fitness, and creates new vectors through latent mutation, interpolation-like crossover, or related variation operators. The Cross-Entropy Method (CEM) instead maintains a sampling distribution over the latent space, evaluates a population of samples, and updates the distribution toward the best-performing elite samples [16]. Covariance Matrix Adaptation Evolution Strategy (CMA-ES) is another population-based continuous optimizer that adapts both the mean and covariance of its search distribution [6]. These methods differ in how they use fitness feedback, but in LSO they share the same requirement: every promising latent candidate must ultimately be decoded into a valid genotype and evaluated by the original objective.

LSO may help with structured genotypes when the latent representation captures properties that are relevant to both syntax and fitness. A compact latent space can reduce the apparent dimensionality of the search, and a regularized decoder can bias generated candidates toward patterns observed in valid genotypes. This can be advantageous when direct random edits are too disruptive. Moreover, continuous search operators can explore directions, mixtures, and local neighbourhoods in a way that has no direct analogue in symbolic genotype strings.

However, these advantages are conditional rather than guaranteed. If the autoencoder reconstructs poorly, many optimized latent vectors may decode into invalid or low-quality genotypes. If the latent space is syntactically valid but not fitness-aware, the optimizer may efficiently explore regions that reproduce common genotype patterns without improving *vertpos*. If the decoder maps large regions of latent space to similar genotypes, search may stagnate because many candidate vectors produce duplicates or near-duplicates. Conversely, if small latent changes decode into unrelated structures, the expected smoothness of the latent space is lost. For this reason, reconstruction quality, validity, diversity, and optimization performance must be analysed separately.

Chapter 3

Methods and Implemented Components

3.1 Dataset and fitness evaluation

The common empirical object used in the **f1** part of the thesis is a dataset of Framsticks **f1** genotype strings paired with scalar fitness values. Each record consists of a genotype written as text and a corresponding *vertpos* value separated from it by a tab character. The dataset contains 10,893 records, and all records provide a finite numeric fitness value. This format preserves a direct link between the symbolic description of an organism and its precalculated value using Framsticks evaluation.

In the implemented data-loading procedure, the genotype string and the fitness label are parsed as two separate fields. The genotype remains the primary structured object: it is later encoded either as a character sequence or through grammar-derived production rules, depending on the model family. The fitness value is stored as a floating-point label associated with the same genotype. For character-level processing, internal whitespace is preserved as part of the input string. For grammar-based processing and benchmark seed preparation, whitespace inside the genotype is normalized away so that the genotype can be parsed and compared consistently. This distinction reflects the fact that the dataset is used both for learning representations of genotype strings and for selecting or analysing examples according to their evaluated fitness.

The stored *vertpos* values should not be interpreted as properties directly computed from raw genotype text. A genotype first has to be accepted by the Framsticks representation, developed into a phenotype, and evaluated in simulation. The *vertpos* criterion is therefore a phenotype-level measurement returned by Framsticks for the organism described by the genotype. This distinction is important for both parts of the thesis. CoSO and Latent Space Optimization operate on genotype representations or propose genotype-level modifications, but their optimization relevance is determined only after the resulting candidates are interpreted and evaluated as Framsticks organisms.

For autoencoder training, the dataset provides examples from which the syntax and structural regularities of **f1** genotypes can be learned. The implementation supports an optional train-validation split rather than relying on a separate validation file. When this split is enabled, a fixed fraction of the parsed examples is assigned to validation using a pseudorandom split controlled by the training seed, while the remaining examples form the training subset. The validation-enabled training scripts use a conventional 0.2 validation fraction. Fitness-aware variants additionally require finite stored fitness labels, because the scalar value can be used as an auxiliary target or as a conditioning signal, but the label remains tied to the original evaluated genotype.

During optimization experiments, dataset fitness values and fresh fitness evaluations play differ-

ent roles. Stored values are suitable for selecting starting genotypes, grouping examples by initial fitness, and providing labels during representation learning or analysis. However, newly produced candidates are not assigned fitness by looking up or estimating their dataset labels. Instead, candidate genotypes generated by direct genotype-level search, CoSO-style modification, or decoding from latent vectors must be evaluated through Framsticks with the same *vertpos* criterion. If a candidate cannot be evaluated successfully, it is treated as invalid for optimization purposes. This procedure keeps the final objective common across the compared methods: regardless of how a candidate is produced, its useful fitness is the true Framsticks evaluation of the phenotype generated from its **f1** genotype.

3.2 Grammar-based representation of **f1** genotypes

The grammar-based part of the implementation represents a Framsticks **f1** genotype as a structured expression rather than only as an unconstrained sequence of characters. The raw genotype is still a variable-length string, but for grammar-aware processing it is first normalized by removing internal whitespace and then tokenized into individual **f1** symbols. The resulting token sequence is accepted only if it can be parsed by the **f1** context-free grammar used in the implementation. This parsing step separates syntactically admissible training examples from strings that cannot be represented as grammar derivations within the configured maximum production-sequence length.

The grammar starts from the nonterminal $\langle S \rangle$ and describes a genotype as a structured sequence of construction elements. Each element contains an optional modifier prefix, the basic structural symbol **X**, and optionally a parenthesized list of child branches. The supported modifier terminals include uppercase and lowercase variants of the main **f1** modifier symbols. The grammar used in the implementation can be summarized as follows. The simplified grammar used by the grammar-aware representation can be written as a context-free grammar

$$G = (N, \Sigma, P, \langle S \rangle),$$

where $\langle S \rangle$ is the start symbol, Σ contains the terminal **f1** symbols, and P contains the following productions:

$$\begin{aligned} \langle S \rangle &\rightarrow \langle Elem \rangle \langle Seq \rangle \\ \langle Seq \rangle &\rightarrow \langle Elem \rangle \langle Seq \rangle \mid \epsilon \\ \langle Elem \rangle &\rightarrow \langle Mods \rangle \mathbf{X} \langle Children \rangle \\ \langle Mods \rangle &\rightarrow \langle Mod \rangle \langle Mods \rangle \mid \epsilon \\ \langle Children \rangle &\rightarrow (\langle BranchList \rangle) \mid \epsilon \\ \langle BranchList \rangle &\rightarrow \langle S \rangle \langle BranchTail \rangle \mid \epsilon \\ \langle BranchTail \rangle &\rightarrow , \langle S \rangle \langle BranchTail \rangle \mid \epsilon \\ \langle Mod \rangle &\rightarrow \mathbf{R} \mid \mathbf{r} \mid \mathbf{Q} \mid \mathbf{q} \mid \mathbf{C} \mid \mathbf{c} \mid \mathbf{L} \mid \mathbf{l} \mid \mathbf{W} \mid \mathbf{w} \mid \mathbf{M} \mid \mathbf{m} \mid \mathbf{I} \mid \mathbf{i} \mid \mathbf{F} \mid \mathbf{f} \mid \mathbf{A} \mid \mathbf{a} \mid \mathbf{S} \mid \mathbf{s} \mid \mathbf{E} \mid \mathbf{e}. \end{aligned}$$

Here ϵ denotes an empty expansion. The grammar captures the subset of **f1** syntax used by the grammar-aware models: a genotype is a non-empty sequence of elements, each element contains an optional modifier prefix followed by the structural symbol **X**, and an element may introduce a parenthesized list of comma-separated child branches. This formalization describes syntactic admissibility for the implemented representation; semantic validity and fitness still depend on Framsticks phenotype construction and simulation.

After parsing, a genotype is encoded as the sequence of productions used in one parse tree. Each production is assigned an integer index, and the production sequence is padded to a fixed maximum length for batch processing. In the integer production-index encoding, positions beyond

the end of the actual derivation use a separate padding index; this index is not one of the grammar productions. The one-hot grammar encoding uses a different convention: its columns correspond only to grammar productions, and padded positions activate the final production column, which corresponds to the grammar’s **Nothing** $\rightarrow$ **None** production. These two related dataset encodings support different model interfaces. The index form is convenient for embedding-based sequence models, whereas the one-hot form is convenient for models that predict a distribution over grammar productions. In both cases, the object being learned is not a character string directly, but a derivation trace that can be expanded back into a genotype string.

Decoding follows the inverse idea. A stack of nonterminals is initialized with the start symbol. At each step, the next predicted production is applied to the current nonterminal, terminal symbols are emitted to the reconstructed genotype, and newly introduced nonterminals are pushed onto the stack in the order required for subsequent expansion. This stack-based procedure gives the production sequence an explicit structural interpretation. It also explains why the order of production choices matters: the same production index can be meaningful only when the nonterminal at the top of the derivation stack has a compatible left-hand side.

The implementation therefore precomputes grammar masks from the set of productions. For every left-hand-side nonterminal, a binary mask identifies exactly the productions whose left-hand side matches that nonterminal. During masked decoding, the current derivation stack determines which nonterminal is being expanded, and the corresponding mask suppresses productions that would be syntactically incompatible at that position. When the stack is empty, the remaining sequence positions are treated as a finished derivation through the designated empty or padding state used by the corresponding encoding. This mechanism does not evaluate the biological or fitness quality of a genotype, but it prevents a decoder from selecting productions that violate the local grammar state.

Several later model variants use the same grammar representation with different amounts of structural context. Grammar-rule models operate on production sequences. Masked variants additionally use the current nonterminal to restrict admissible production choices during decoding. Tree-oriented variants reconstruct the left-hand-side context implied by the derivation stack and can provide this context to the encoder or decoder. The LHS-conditioned variants make this context explicit by embedding the currently expanded nonterminal, while LHS-depth variants also represent the current stack depth. These details belong to the autoencoder section at the architectural level, but at the representation level they all rely on the same principle: an **f1** genotype is treated as a grammar derivation with a current expansion state, rather than as an unstructured string.

This representation is useful for Latent Space Optimization because latent search ultimately proposes candidates through a decoder. If the decoder is allowed to generate arbitrary symbols, many latent points can map to strings that do not respect the **f1** syntax and therefore cannot be meaningfully evaluated. A production-based representation narrows the decoding problem to choosing grammar rules, and grammar masks further constrain these choices to the rules allowed by the current derivation context. Syntactic validity is not sufficient for high *vertpos* fitness, and it does not guarantee successful optimization, but it is a prerequisite for using decoded candidates as Framsticks genotypes in later evaluation and search procedures.

3.3 Tree-based fragment handling

The CoSO component operates on textual Framsticks **f1** genotypes, but substitutions are not selected by cutting the string at arbitrary character positions. Before fragments are considered

for replacement, the genotype is interpreted as a tree-like structure induced by the **f1** notation. This representation reflects the fact that an **f1** genotype contains nested branches and sequences of construction elements, so many syntactically meaningful fragments correspond to complete substructures rather than to unconstrained substrings.

The purpose of this tree-based handling is to define candidate cut points that are likely to preserve the syntactic organization of the genotype. A substitution performed at such boundaries replaces a coherent fragment, for example a branch or a sequence of elements, instead of splitting a modifier sequence, parenthesized branch, or separator structure in the middle. This does not make every substitution valid, because the surrounding context may still require additional syntactic adjustments, but it removes most of the errors that would result from purely character-level cutting. In practice, the resulting candidate genotype is still passed through the Framsticks repair mechanism before evaluation, so minor syntactic inconsistencies such as missing separators or unmatched structural markers may be corrected when possible.

The same tree-based view also provides the pool of fragments that may later be reused as substitutions. Each parsed genotype contributes a set of coherent substructures that can be compared with fragments from other genotypes. Therefore, the fragment extraction stage plays two roles: it identifies the locations in a recipient genotype where a substitution may be attempted, and it supplies donor fragments that can be stored and reused when a compatible context is encountered later in the run.

3.4 Compatible Substitutions Optimization module

The implemented CoSO module performs optimization directly in the **f1** genotype space. It does not introduce an intermediate continuous representation and does not train a model before the search. Instead, it maintains a collection of previously observed genotype fragments and uses them to propose context-aware substitutions in currently processed genotypes. The final objective remains the true Framsticks evaluation of the decoded organism with the *vertpos* criterion.

At the beginning of a run, the module initializes the Framsticks evaluation environment, random seeds, population parameters, archive parameters, compatibility thresholds, and the logging procedure. Starting genotypes can be obtained either from a dataset of **f1** genotypes with stored fitness values or generated through the Framsticks mutation mechanism. Dataset fitness values are used when the exact genotype is already known, whereas newly produced candidates are evaluated through Framsticks. Empty genotypes and genotypes exceeding the configured length constraint are treated as uncompetitive, and all accepted evaluations update the best solution observed so far.

The search is organized around an active genotype that is repeatedly improved by compatible substitutions. For this genotype, the tree-based fragment handling described above produces candidate recipient fragments together with their locations in the original genotype. Each fragment is described by a small set of representation-level features. In the final **f1** implementation, these features characterize the spatial effect of the fragment and the direction associated with its terminal part. A pair of fragments is considered compatible when the distance between each corresponding feature descriptor does not exceed its assigned threshold. The spatial threshold controls how similar the displacement induced by two fragments must be (interpreted in the coordinate units of the generated Framsticks model), whereas the directional threshold controls how similar their terminal directions must be (feature is a unit direction vector, so Euclidean distance corresponds approximately to angular deviation). Lower thresholds make the archive retrieval stricter and return only near-identical fragments; higher thresholds make retrieval more permissive and allow

substitutions between less similar fragments. The intent is to compare fragments by the structural role they may play in the constructed phenotype, rather than by their raw textual form or absolute position in the genotype.

The archive stores donor fragments indexed by these feature values. When a genotype can no longer be improved, its fragments are inserted into the archive together with the fitness of the solution from which they were obtained. For each feature bucket, only a limited number of entries is retained, with preference given to fragments originating from better-performing genotypes. The archive therefore acts as a memory of building blocks discovered during the search, grouped by approximate compatibility rather than by genotype position.

During improvement, the module queries the archive for fragments whose feature values are close enough to the feature values of a recipient fragment. The accepted distance is controlled by the spatial and directional compatibility thresholds. Retrieved donor fragments are then tested as possible replacements. Before a candidate is evaluated, overly large substitutions and substitutions that would violate the maximum genotype length are rejected. This prevents a compatible feature match from replacing too much of the current genotype or producing candidates outside the intended representation constraints.

For each remaining candidate, the selected recipient fragment is replaced with the donor fragment in the full genotype. The candidate is then passed to the Framsticks repair mechanism. This repair step is important because feature compatibility and tree-based cut points reduce syntactic disruption but do not guarantee that every resulting string is directly accepted by Framsticks. Candidates that cannot be repaired or evaluated are discarded. Candidates that can be evaluated are accepted only if their *vertpos* value is strictly higher than the fitness of the genotype before the substitution. Thus, as in optimal mixing, a fragment is considered useful only after it has been tested in the complete recipient context.

If an accepted substitution is found, the improved genotype becomes the new active genotype and the process continues from this modified solution. If no compatible substitution improves the active genotype, the solution is treated as locally converged. Its fragments are added to the archive, the solution may be inserted into the elite set, and a new active genotype is selected or generated. The elite set is kept limited to the best solutions encountered during such convergence events. This creates a restart-like process in which later genotypes are improved using an archive that accumulates fragments from earlier search attempts.

The module records the main events of the run in a JSONL log. The stored events include metadata, new starting genotypes, accepted improvements, archive retrieval statistics, convergence events, progress snapshots, final elite solutions, and the best genotype found. These logs are used later to reconstruct optimization curves and analyse how the archive-based substitution process behaves over time. The run terminates when the configured number of full Framsticks evaluations is reached.

3.5 Autoencoder-based genotype representation

The autoencoder-based representation used in the Latent Space Optimization part of the thesis maps a variable-length **f1** genotype to a fixed-dimensional numerical vector and then reconstructs a genotype from that vector. In all variants, the encoder is responsible for transforming an input sequence into a latent representation, while the decoder is responsible for producing a sequence that can be interpreted as a genotype. This representation is not a replacement for the Framsticks genotype itself. It is an intermediate search representation: optimization may operate on the latent

vector, but every candidate must eventually be decoded back to **f1** text before it can be evaluated with the true *vertpos* criterion.

Two input families are used. The first family treats the genotype as a character sequence. A character-level dataset builds a vocabulary containing padding, start, and end symbols, followed by the characters observed in the training strings. Each genotype is encoded as a padded sequence of token indices. The corresponding character-level variational autoencoder uses recurrent sequence encoders and decoders: the encoder embeds the tokens and summarizes the sequence into the parameters of a latent distribution, and the decoder generates output tokens autoregressively from a sampled latent vector. This variant leaves syntactic regularities to be learned from examples, because it does not explicitly know which character choices are legal in a given grammatical context.

The second family uses the grammar representation introduced in Section 3.2. A parsed genotype is encoded as a sequence of production-rule indices, optionally represented in one-hot form. The reconstruction target is therefore not directly a sequence of raw characters, but a derivation trace that can be expanded into an **f1** string. This distinction is important for validity. If the decoder predicts a rule sequence that follows the derivation stack consistently, the resulting output is constrained by the same grammar that was used to encode the training examples. The decoder output is then converted to genotype text by applying predicted productions to the current nonterminal and emitting terminal symbols.

The implemented grammar-rule variational autoencoder follows the standard encoder-decoder pattern at the level of production indices. The encoder embeds the production sequence and maps it to mean and log-variance vectors. During training, a latent vector is sampled by the reparameterization mechanism, and the decoder is trained to reproduce the input production sequence. The objective combines a reconstruction term over sequence positions with a Kullback-Leibler regularization term, with a scheduled weight on the regularization term during training. Teacher forcing is used for recurrent decoders during training and is reduced over the training schedule. At the methods level, the relevant point is that the model learns a continuous latent code from which complete grammar-rule sequences can be generated.

Masked grammar variants add explicit syntactic constraints to decoding. The **f1** grammar provides, for each left-hand-side nonterminal, the set of productions that may expand it. During masked decoding, the current derivation stack determines the active nonterminal, and logits for incompatible productions are suppressed. This mechanism does not guarantee that a decoded genotype has high fitness, and it does not evaluate semantic properties of the organism. It only enforces local grammatical compatibility during rule selection, which reduces the number of syntactically impossible production choices available to the decoder.

Tree-oriented variants use the fact that a production sequence implicitly defines a derivation tree. In these models, the current left-hand-side context is reconstructed from the stack of nonterminals generated by previous rules and is provided as an additional input to the encoder. Masked tree variants combine this tree-aware context with rule masks in the decoder. LHS-conditioned masked tree variants go further by giving the decoder an embedding of the currently expanded nonterminal, and LHS-depth variants additionally represent the current stack depth. These mechanisms all use the same grammar as Section 3.2, but they expose more of the derivation state to the neural model instead of requiring it to infer the state only from preceding rule indices.

Conditional extensions are also implemented for selected tree-oriented models. Fitness-conditioned variants append a scalar conditioning value to the latent vector before decoding. Structural conditional variants use a small vector of auxiliary structural descriptors rather than only a scalar fitness condition. These variants are methodological extensions of the same autoencoder design: the latent vector remains the main representation, while the condition supplies additional informa-

tion to the decoder. The presence of a conditioning signal should not be interpreted as replacing true fitness evaluation, because decoded candidates are still evaluated through Framsticks during optimization.

A Transformer-based grammar variational autoencoder is included as an alternative to recurrent sequence modelling. It uses embeddings of production-rule indices together with positional encodings and Transformer encoder-decoder blocks. The encoder pools non-padding positions to obtain latent distribution parameters, and the decoder generates a production sequence from the latent vector using a causal decoding structure. This variant keeps the same grammar-rule input and reconstruction target as other grammar-based models, but changes the sequence model used to capture dependencies between production choices.

Finally, a vector-quantized grammar autoencoder is implemented as a related representation-learning variant. Instead of sampling from a continuous variational distribution, it maps an encoded production sequence to a latent vector that is quantized through a learned codebook and then decoded back into production rules. This variant is relevant as an alternative autoencoding mechanism for grammar-rule sequences, although the main Latent Space Optimization procedures considered in this thesis operate on continuous latent vectors generated by variational models.

Across these model families, the training target remains reconstruction of the input representation: characters for the character-level model and grammar production sequences for grammar-aware models. Reconstruction, however, is only a prerequisite for later optimization. A model that reconstructs valid examples well may still define a latent space that is difficult to optimize, and a syntactically constrained decoder may still produce organisms with low *vertpos*. For this reason, autoencoder training and latent-space search are treated as separate methodological components: the autoencoder defines the representation, while the optimization methods described next decide how that representation is explored.

3.6 Latent-space optimization methods

Latent-space optimization uses a trained genotype autoencoder as a generator of candidate **f1** genotypes. The optimizer itself manipulates fixed-dimensional latent vectors, but the objective value assigned to a vector is obtained only after decoding. In the implemented latent EA, CEM, and CMA-ES benchmark procedures, latent candidates are decoded as grammar-rule sequences, converted into genotype text, checked for syntactic admissibility, and evaluated in Framsticks. Character-level autoencoders remain part of the broader representation set described in Section 3.5, but they are not the decoding context used by these grammar-rule optimization loops. The scalar feedback used by the optimizer is the resulting true *vertpos* fitness, with invalid or unevaluable candidates treated as uncompetitive. Thus, the learned model proposes candidates, whereas Framsticks remains the source of fitness values.

In the seeded setting used by the implemented procedures, optimization starts from one or more existing genotypes. A starting genotype is encoded by the autoencoder, and the mean vector produced by the encoder is used as the initial latent point. This choice gives the search a concrete reference candidate rather than beginning only from an unconditional prior sample. For grammar-rule models, starting genotypes must be encodable by the grammar representation and must fit within the configured maximum production-sequence length. The decoded form of the starting latent vector can also be evaluated, because the autoencoder reconstruction of the seed may differ from the original genotype.

All latent-space methods share the same evaluation loop. A population or distribution produces a batch of latent vectors. Each vector is decoded to an **f1** genotype, normalized for comparison,

and rejected if it is empty, syntactically invalid, duplicated when uniqueness is required, or impossible to evaluate. Remaining candidates are evaluated by Framsticks with the *vertpos* criterion. Fitness values are cached for already evaluated genotypes so that the optimizer is not rewarded for repeatedly generating the same string. The best decoded genotype and its true fitness are tracked separately from the latent vector that produced it, because the final output of the method is an **f1** genotype rather than the vector itself.

The latent-space evolutionary search maintains a population of individuals represented by both decoded genotype and latent vector. Initial population members are produced by perturbing the encoded seed vector with Gaussian noise of selected standard deviations and evaluating the decoded genotypes. In each generation, parents are selected according to true evaluated fitness. Offspring latent vectors are generated through several continuous-space operators: mutation by Gaussian perturbation, crossover-like interpolation between parent vectors, optional extrapolation along lines between parents, and directional moves biased by the difference between a high-fitness vector and another selected vector. After decoding and true evaluation, valid offspring are merged with elite survivors, and the population is truncated to the configured size.

Random immigrants are supported in the latent evolutionary search as an exploration mechanism. With a configurable probability, a child vector is sampled from a standard normal distribution rather than produced from selected parents. This operation introduces latent candidates that are not constrained to remain near the current population. Methodologically, it reduces the risk that the population collapses prematurely around a narrow decoded region, although the sampled vectors must still decode to valid and useful genotypes to affect the search.

The same evolutionary implementation can optionally re-encode accepted offspring. In this variant, a decoded genotype that passes validity and evaluation is encoded again, and the resulting latent mean replaces the raw child vector. Re-encoding projects the individual back through the autoencoder’s representation of the decoded genotype. This can make the stored latent population more consistent with the model’s encoder–decoder manifold, but it is an operator choice rather than a separate objective function.

The Cross-Entropy Method (CEM) is implemented as a distribution-based optimizer over latent vectors. Starting from the encoded seed, the method maintains a diagonal normal sampling distribution. At each iteration, it samples a population of latent candidates, decodes and evaluates them, selects an elite fraction according to *vertpos*, and updates the distribution mean and standard deviation toward the elite set. Smoothing and a minimum standard deviation are used so that the distribution does not change too abruptly or collapse completely. CEM therefore differs from the latent evolutionary search in that it does not maintain explicit genotype individuals across generations; it adapts a sampling distribution from elite latent samples.

CMA-ES is implemented as another continuous distribution-search method. It samples latent candidates around a mean vector, evaluates the decoded genotypes, and updates the search mean and covariance structure according to the best-performing samples. In this setting, the optimizer minimizes the negative of the true fitness only implicitly through the scores returned after decoding and Framsticks evaluation; it does not use gradients through the decoder or through the simulator.

The latent-space methods are therefore derivative-free black-box optimizers with respect to the final objective. The autoencoder is trained before optimization and is used only to map between **f1** genotypes and latent vectors. During search, no learned surrogate fitness is used to replace Framsticks evaluation, and no gradient of *vertpos* is available. This separation is essential for the comparison performed in later chapters: differences between methods can be attributed to how they explore the learned representation and how often they produce valid, diverse, high-fitness decoded genotypes, not to a change in the underlying fitness criterion.

3.7 Baseline evolutionary methods

The baseline evolutionary method provides a non-learned reference point for the latent-space procedures. It operates directly on normalized Framsticks `f1` genotype strings and uses the standard variation operators exposed by the Framsticks library. No autoencoder, latent vector, learned surrogate, or fragment archive is involved. As in the other optimization procedures, the objective value assigned to a candidate is the true Framsticks *vertpos* value obtained after the candidate genotype is interpreted and simulated.

Each run starts from one source genotype selected in the benchmark manifest. The source genotype is inserted into the initial population together with its stored manifest fitness. The remaining initial individuals are generated by repeatedly applying the Framsticks mutation operator to the source genotype. Each mutant is normalized, checked for duplication against the evaluation cache, and evaluated through Framsticks. Empty, duplicate, failed, or invalid candidates are discarded. This produces an initial population around the same starting organism while keeping the original source genotype available as a possible survivor.

The generational loop follows a conventional black-box evolutionary pattern in genotype space. Parent candidates are selected from the current population by tournament selection with tournament size three. For each offspring attempt, the implementation samples a small pool of tournament winners. With probability 0.25, two selected parents are combined using the Framsticks crossover operator. A crossover result is then mutated with probability 0.25. If crossover is not selected, one selected parent is mutated directly. The resulting genotype is normalized and evaluated only if it is non-empty and has not already appeared in the run-level cache.

After offspring generation, the previous population and newly accepted children are merged. Survival is elitist at the level of evaluated genotypes: duplicate normalized strings are collapsed, the best score associated with each genotype is retained, and the population is truncated to the highest-fitness individuals. Thus, strong previously discovered genotypes remain in the population unless enough better offspring are found. The recorded history stores the best score, number of unique scored genotypes, current population size, accepted child count, and attempted child-generation count for every generation.

The final benchmark configuration uses 300 generations, population size 100, requested offspring size 100, elite-size parameter 10, tournament size three, Framsticks crossover probability 0.25, mutation-after-crossover probability 0.25, attempt factor 30, and base pseudorandom seed 777. No explicit maximum-evaluation or wall-clock limit is imposed in the final baseline jobs. A completed run therefore has a nominal budget of about 30,000 offspring slots plus initialization, although the exact number of unique scored genotypes depends on duplicates, invalid candidates, and retry limits.

Two baseline execution variants use the same evolutionary loop and the same starting manifest. The standard baseline evaluates candidates with the chain `eval-allcriteria.sim`, `deterministic.sim`, `sample-period-2.sim`, `only-body.sim`. The mini baseline replaces the first file with `eval-allcriteria-mini.sim` and keeps the remaining deterministic, sampling-period, and body-only settings. These variants should therefore be interpreted as direct genotype-space evolutionary references under their recorded Framsticks simulation configurations.

Methodologically, this baseline answers a different question from the latent-space optimizers. It measures what can be achieved by applying native Framsticks mutation and crossover directly to `f1` strings under a similar population-and-generation schedule. The latent methods instead test whether a learned continuous representation helps propose useful decoded genotypes. Comparing them therefore requires preserving the distinction between the search representation, the candidate-

generation operators, and the exact simulation configuration used for evaluation.

3.8 Implementation architecture

The implemented Latent Space Optimization system is organized as a sequence of components with explicit interfaces between symbolic representation, neural inference, black-box optimization, and simulation. This separation allows the representation model and the optimizer to be varied independently. A candidate is always transferred between components in one of three forms: F1 genotype text, a grammar-production sequence, or a continuous latent vector. Only genotype text is submitted to Framsticks, and only the score returned by Framsticks is used as the final optimization objective.

The data and grammar component reads genotype–fitness records, normalizes genotype text where grammar processing requires it, and converts accepted examples into either character indices or grammar-production indices. For grammar-aware models, the same component provides production dictionaries, padding conventions, derivation context, and masks of rules compatible with the current nonterminal. It is used both when constructing training batches and when encoding a starting genotype for optimization. This shared representation layer prevents training and search from using incompatible interpretations of an F1 string.

The learning component creates the selected autoencoder from a recorded configuration and trains it on the corresponding dataset encoding. It records model dimensions, training hyperparameters, validation diagnostics, and periodic or validation-selected checkpoints together with the model state. A checkpoint and its associated configuration form the input to later inference: the model architecture and dataset interface are reconstructed before the learned parameters are loaded. Training is kept outside the optimization loop, so benchmark runs do not update the autoencoder in response to candidate fitness.

The inference component exposes encoding and decoding operations independently of training. Encoding parses and pads a genotype, applies the appropriate encoder, and returns the latent mean used as a deterministic starting point. Decoding maps one or more latent vectors to production choices, expands these choices into genotype text, and applies the representation-specific syntactic checks. Batched decoding is used where an optimizer proposes many vectors at once. This interface hides architecture-specific calls from the search procedure while retaining a common decoded-genotype output.

The fitness-evaluation component resolves a simulation configuration, submits normalized genotypes to Framsticks, extracts the requested *vertpos* value, and maps failed evaluations to an invalid outcome. Optimization procedures maintain genotype-score caches around this interface to avoid assigning fitness repeatedly to the same normalized string. The evaluator is deliberately independent of both the autoencoder and the optimizer; consequently, a learned fitness head may be inspected as a training diagnostic but cannot replace true Framsticks evaluation in the final search path.

Above these components, the optimization layer implements the latent-space EA, CEM, and CMA-ES through a common decode–validate–evaluate cycle for generated candidates. Each optimizer receives a fixed checkpoint context, an encoded starting genotype, a pseudorandom generator state, and a search budget. Their initialization and raw return semantics are not identical: latent EA includes the original genotype with its stored manifest fitness, while CEM and CMA-ES return the best freshly evaluated decoded candidate from their latent distributions. The recorded output therefore preserves both the method-dependent raw score and a derived score obtained by retaining the original genotype whenever the raw result does not improve its stored label. The system

can exchange an optimizer without modifying genotype parsing, checkpoint loading, decoding, or simulation evaluation, while the analysis layer makes this source-retention policy explicit.

Experiment planning and execution form a separate orchestration layer. A manifest defines starting genotypes and their initial-fitness buckets, while a plan enumerates fixed checkpoint–optimizer–bucket configurations and their output artifacts. Each planned unit records its configuration and seeds, writes per-starting-genotype summaries and histories, and preserves the simulation settings used for evaluation. Aggregation then validates expected result counts, joins records only by declared identifiers, and computes configuration-level or bucket-level summaries. Keeping planning, execution, and aggregation separate makes it possible to audit whether an apparent comparison was produced under a common evaluation condition before interpreting it as controlled evidence.

Chapter 4

Experimental Design

4.1 Research questions

The experiments are designed to evaluate how two representation-aware optimization strategies behave when applied to structured, variable-length genotypes. The questions are formulated at the level of the general optimization problem, while the empirical answers are obtained using Framsticks **f1** genotypes and the *vertpos* criterion as the common benchmark.

RQ1. How reliably do the compared methods produce valid and evaluable genotypes?

RQ2. To what extent do the compared methods improve the fitness of starting genotypes?

RQ3. How does explicit context-aware substitution perform in comparison with learned latent-space search?

RQ5. How sensitive are the results to starting conditions and initial fitness ranges?

4.2 Compared methods

The LSO experiments vary two factors that must be distinguished conceptually. The autoencoder checkpoint defines the learned mapping between an **f1** grammar derivation and a continuous latent vector, whereas the optimizer defines how vectors in that space are proposed and selected. A method configuration is therefore a fixed checkpoint–optimizer pair rather than an autoencoder alone. The final design does not select a different checkpoint or optimizer for each starting genotype.

The representation-learning stage covered character, grammar-rule, masked grammar, tree-oriented, LHS-conditioned, LHS-depth, Transformer, fitness-aware, and vector-quantized families. The final continuous-space benchmark uses five grammar-rule VAE checkpoints selected from this broader set. TreeVAE-80 and TransformerVAE-90 use 32-dimensional latent spaces and represent two different sequence-modelling architectures. The LHS latent-256 checkpoint uses a masked tree decoder conditioned on the current left-hand-side nonterminal and a 256-dimensional latent space. The LHS-depth checkpoint adds derivation-depth context and uses 128 latent dimensions. The fitness-aware LHS checkpoint also uses 128 dimensions; it was trained with an auxiliary latent fitness-prediction loss, but its auxiliary head is not used as the objective or as a surrogate during final optimization. These entries are fixed learned representations, not five optimization algorithms.

Each checkpoint is combined with three derivative-free latent optimizers. The latent-space EA maintains decoded individuals and latent vectors, uses tournament selection and elitist survival, and proposes offspring through Gaussian mutation, interpolation or extrapolation between parents,

directional moves, and occasional random immigrants. CEM instead maintains a diagonal normal distribution and updates it from an elite fraction of each sampled population. CMA-ES adapts a multivariate search distribution, including covariance information, from decoded candidate scores. All three methods use the latent mean of the same source genotype to locate the initial search region and receive true Framsticks *vertpos* feedback for generated candidates. Their treatment of the source itself differs. Latent EA inserts the original genotype into the initial population with its stored manifest fitness and can retain it through elitist survival. CEM and CMA-ES centre their initial distributions at the encoded mean, decode the sampled vectors (including the current mean), and evaluate those decoded strings; their raw best score does not include an automatic fallback to the original genotype’s stored label. Comparative analyses therefore apply the uniform retained-source decision policy defined in Section 4.4 rather than treating the three raw outputs as semantically identical.

The CoSO condition represents the direct genotype-space counterpart to the latent-space methods. It does not use a learned checkpoint and does not optimize an intermediate continuous representation. Instead, it operates on **fl** strings themselves by extracting coherent tree-based fragments, describing them with representation-level compatibility features, and attempting substitutions between fragments whose descriptors are sufficiently similar. A CoSO method configuration is therefore defined by the fragment-handling procedure, the compatibility thresholds, the archive settings, and the search budget, rather than by a checkpoint-optimizer pair.

In the evaluated CoSO configuration, candidate substitutions are guided by two fragment descriptors: the spatial effect of a fragment and the direction associated with its terminal part. Previously observed fragments are stored in an archive indexed by these descriptors and are later retrieved as possible replacements for fragments in the currently processed genotype. Each proposed replacement is inserted into the full recipient genotype, optionally repaired by Framsticks, and evaluated with the true *vertpos* criterion. The substitution is accepted only when it improves the current genotype. CoSO is therefore treated as an explicit context-aware substitution method, whose search space remains the original discrete **fl** representation.

The treatment of the source genotype in CoSO is closer to a direct local-improvement procedure than to the distribution-based latent optimizers. The active genotype is evaluated as a complete **fl** candidate, then repeatedly modified through accepted substitutions. If no improving substitution is found, the genotype is treated as locally converged, its fragments may contribute to the archive, and the search proceeds from another starting genotype. Consequently, the raw CoSO best score reflects mostly the semantics of the best genotype actually retained during direct search in the original representation.

Two benchmark variants of the same genotype-space Framsticks EA provide baseline conditions. Both use Framsticks mutation and crossover, tournament selection, elitist survival, population and offspring sizes of 100, and 300 generations from each manifest genotype. They differ in the first simulation file used to score generated candidates. The standard baseline uses the same four-file chain as the LSO benchmark: `eval-allcriteria.sim`, `deterministic.sim`, `sample-period-2.sim`, and `only-body.sim`. The mini baseline replaces the first file with `eval-allcriteria-mini.sim` and retains the other three. They are consequently treated as two separate baseline conditions, with the standard variant serving as the same-chain baseline for LSO and the mini variant as the closest simulation-chain reference for CoSO.

4.3 Benchmark setup

The common benchmark is centered on Framsticks **f1** genotypes and on the *vertpos* fitness criterion introduced in Section 3.1. The dataset is used to provide existing genotypes with known starting fitness values, but it is not used as a lookup table for the fitness of newly generated candidates. During optimization, each candidate produced by a baseline evolutionary run or by decoding a latent vector is converted to an **f1** genotype and evaluated through Framsticks with the same *vertpos* criterion. Candidates that cannot be represented or evaluated successfully are treated as invalid for the corresponding search procedure.

Starting genotypes for the final benchmark are selected from six initial-fitness buckets. The bucket ranges are $[0.30, 0.50)$, $[0.50, 0.75)$, $[0.75, 1.00)$, $[1.00, 1.20)$, $[1.20, 1.50)$, and $[1.50, 2.30)$. The upper bound of the final bucket was chosen to include the fittest available starting genotype, which had an initial fitness score of about 2.30. Twenty-five starting genotypes are sampled from each bucket, giving 150 starting points in total. The same seed manifest is used for all latent-space configurations and for the Framsticks EA baselines. This provides identical starting genotypes and permits seed-level alignment between LSO and the standard baseline. The mini baseline remains a separate simulation condition because it begins with `eval-allcriteria-mini.sim`.

The CoSO evaluation is divided into two experimental stages. The first stage is a compatibility-threshold selection experiment. It uses genotypes sampled from the available **f1** dataset and evaluates several pairs of spatial and directional compatibility thresholds. The purpose of this stage is not to define a separate final comparator for LSO, but to identify which threshold pair gives the most promising behaviour for the archive-based substitution mechanism. In this experiment, CoSO is analysed as a direct genotype-space optimizer whose performance depends on how restrictive or permissive the fragment-matching criterion is.

The second CoSO stage uses the selected threshold pair in a bucket-aligned benchmark. This stage follows the same initial-fitness bucket structure as the latent-space benchmark: starting genotypes are grouped by their stored source *vertpos* values so that the method can be analysed separately for weak, medium, and already strong initial candidates. A CoSO run operates directly on **f1** strings, extracts tree-based genotype fragments, stores previously observed fragments in a compatibility-indexed archive, and tests substitutions in complete genotypes. Newly produced candidates are evaluated through Framsticks with the *vertpos* criterion.

Unlike the latent benchmark, CoSO does not form checkpoint–optimizer combinations. A CoSO configuration is defined by its compatibility thresholds, population and elite sizes,

and evaluation budget defining stop point of an algorithm. The threshold-selection stage explores part of this configuration space, while the bucket-aligned stage fixes the selected threshold pair and focuses on the dependence of CoSO performance on the initial fitness range.

The final Latent Space Optimization benchmark combines five selected autoencoder checkpoints with three latent-space optimizers. The checkpoint labels used in the benchmark are:

- 09_tree_vae_epoch80,
- 10_transformer_vae_epoch90,
- 02_lhs_latent256_seed42_best,
- 06_lhs_depth_seed42_best,
- fitness_aware_lhs_seed42.

For each checkpoint, three latent methods are evaluated: the latent-space evolutionary algorithm, reported as `latent_ea`; the Cross-Entropy Method, reported as `cem`; and CMA-ES, reported as `cmaes`. The resulting latent part of the benchmark therefore consists of all combinations of five checkpoints, three methods, six buckets, and twenty-five seeds per bucket.

The baseline part of the benchmark uses the standard Framsticks EA from the same starting genotypes. There is one baseline run for each of the 150 starting genotypes. The nominal search budget is kept consistent at the design level: evolutionary runs use 300 generations with population size 100, while distribution-based latent methods use 300 iterations with the same population size. Actual numbers of unique evaluated genotypes may differ between runs because invalid candidates, duplicates, and cached evaluations are handled by the optimization procedure. The number of unique evaluated genotypes is recorded for all runs, while invalid and duplicate counts are interpreted only for methods and jobs that record them; these measurements are used for later metric analysis rather than being treated as separate benchmark conditions.

The computational execution separates neural latent-space jobs and baseline evolutionary jobs because latent decoding uses trained autoencoder checkpoints, whereas the Framsticks EA baseline does not require neural inference. In the completed artifacts, LSO and the standard CPU baseline use the same four-file chain: `eval-allcriteria.sim`, `deterministic.sim`, `sample-period-2.sim`, and `only-body.sim`. The mini CPU baseline uses the corresponding four-file chain beginning with `eval-allcriteria-mini.sim`. All these runs target *vertpos* and reuse the same starting-genotype manifest; only the mini baseline is a separate simulation condition from the standard LSO–baseline chain.

4.4 Evaluation metrics

The evaluation uses three connected groups of metrics: representation-learning quality, optimization quality, and search behaviour. This organization reflects the structure of the study. Autoencoder training is not treated only as compression of genotype strings: selected variants also attempt to organize the latent representation with respect to fitness-related information, for example through auxiliary fitness prediction or conditioning. Final optimization quality is nevertheless always measured with the true Framsticks *vertpos* evaluation, not with a neural prediction.

For autoencoder training and checkpoint selection, the main reconstruction metric is validation exact reconstruction accuracy. A reconstruction is counted as exact when the decoded output string is identical to the corresponding reference genotype string after the representation-specific decoding step. Because exact equality is strict for variable-length structured genotypes, average string similarity is also recorded. Selected checkpoints are additionally characterized by unconditional generation diagnostics: syntactic validity of decoded **f1** strings, uniqueness, duplicate rate, novelty relative to the available dataset, and, when evaluated, the distribution of generated *vertpos* values. For fitness-aware variants, fitness-prediction error and correlation are used as auxiliary diagnostics describing whether the learned representation captures fitness-related structure.

The main optimization endpoint is the best *vertpos* value obtained by a method or configuration. For single-source LSO and baseline runs, the benchmark records a raw best-score field

$$F_{m,s}^{\text{raw}},$$

where m denotes the method or configuration and s denotes the starting genotype. Because the optimizers handle the source genotype differently, cross-optimizer summaries use a uniform retained-source decision policy:

$$F_{m,s}^{\text{retained}} = \max(F_{m,s}^{\text{raw}}, F_s^{\text{source}}),$$

with retained improvement

$$\Delta_{\text{source}}^{\text{retained}}(m, s) = F_{m,s}^{\text{retained}} - F_s^{\text{source}}.$$

This metric corresponds to the practical decision made after optimization: keep the generated result if it improves the starting genotype, otherwise retain the source genotype. It makes latent EA, CEM, CMA-ES, and the baseline comparable under one reported endpoint, even though their raw optimizer outputs have slightly different source-handling semantics.

When two single-source methods use the same starting manifest and simulation chain, results are compared directly by starting genotype. For a method or configuration m , the paired difference against the standard Framsticks EA baseline is

$$\Delta_{\text{EA}}(m, s) = F_{m,s}^{\text{retained}} - F_{\text{Framsticks EA},s}^{\text{retained}}.$$

Positive values indicate wins against the baseline, negative values indicate losses, and exact zeros indicate ties. Aggregates over starting genotypes include mean, median, maximum, standard deviation, retained improvement, paired difference, and win rate. Since the completed LSO runs and the standard baseline use the same four-file simulation chain and the same 150 starting genotypes, this comparison is treated as the main same-condition LSO–baseline comparison.

CoSO is evaluated with compatible quality metrics but a different experimental unit. A CoSO run is an archive-based multi-start search over a genotype pool, not a one-run-per-source trajectory. Its primary endpoint is therefore the best *vertpos* value found during the run, together with improvement over the best unmodified genotype introduced into that run:

$$\Delta_{\text{subst}}^{\text{CoSO}}(r) = F_{\text{CoSO},r}^{\text{best}} - F_r^{\text{introduced}}.$$

This makes CoSO comparable with LSO via the baselines at the bucket, configuration, and aggregate-result level, while preserving the fact that it is not a paired per-source method.

Search behaviour and stability are described by quantities recorded during each run. The number of unique evaluated genotypes measures how many distinct genotype strings were assigned fitness values under the nominal budget. Invalid-candidate and duplicate-candidate counts describe how much search effort was lost to unusable or repeated candidates, where such fields are recorded. Best-so-far histories over generations, iterations, or evaluation steps are used to show convergence behaviour and stability over time.

Optimization metrics are also analysed conditionally on the initial-fitness buckets defined in Section 4.3. Bucket-conditioned summaries report final scores and improvements separately for weak, medium, and strong starting material. This shows whether a method mainly improves low-fitness candidates, refines already promising ones, or depends strongly on the initial quality of the genotype pool.

4.5 Experimental protocol

Autoencoder training and final optimization are conducted as separate stages. Training uses the grammar-encodable **f1** data and records the configuration, periodic checkpoints, and reconstruction diagnostics. Validation-enabled runs reserve 20% of the available examples, giving 2,178 validation items in the recorded runs. Split membership is run-specific: configurations with a fixed training seed use a reproducible split, while identical membership is not assumed for runs whose configuration does not specify a seed. Exact reconstruction and average string similarity are evaluated by deterministic decoding from the latent mean. The retained checkpoint set is then subjected to

a separate deterministic reconstruction pass and to unconditional generation diagnostics. These measurements support checkpoint characterization but are not optimization results.

The checkpoint-selection stage produced a frozen list before the final benchmark plan was executed. The ten-checkpoint diagnostic set contains validation-selected checkpoints for the masked and LHS families and explicit epoch checkpoints for TreeVAE-80 and TransformerVAE-90. The final optimization plan fixes five labels: TreeVAE-80, TransformerVAE-90, LHS latent 256, LHS with depth, and the auxiliary fitness-aware LHS model. For every starting genotype, each label is evaluated separately with latent EA, CEM, and CMA-ES. No result-dependent checkpoint switching, optimizer switching, or per-seed portfolio selection is part of a benchmark run.

Starting points are defined by one manifest sampled with pseudorandom seed 20260607. It contains 25 dataset genotypes in each of the six ranges from Section 4.3, for 150 source genotypes in total. Within the final execution, pseudorandom optimizer states are derived deterministically from a fixed base seed of 777 and the planned configuration and source-seed identifiers. This procedure makes repeated ‘of a declared unit reproducible without implying that different algorithms consume identical random-number streams.

Each latent-space EA run uses 300 generations, a population of 100, 100 requested offspring per generation, ten elites, and tournament size three. Its fixed operator settings use mutation standard deviations 0.12, 0.25, 0.50, and 0.90; crossover, extrapolation, directional-move, and random-immigrant probabilities of 0.45, 0.20, 0.25, and 0.05; and no offspring re-encoding in the final plan. CEM and CMA-ES use 300 iterations and populations of 100. CEM uses an elite fraction of 0.25, initial standard deviation 0.35, smoothing 0.35, and minimum standard deviation 0.02; CMA-ES starts with scale 0.35. The Framsticks EA baselines use populations and offspring sets of 100 for 300 generations, ten elites, tournament size three, crossover probability 0.25, and probability 0.25 of mutation after crossover. These settings define one fixed configuration per run.

Initialization follows the method distinction introduced in Section 4.2. Latent EA creates its initial population with the unchanged source genotype as one individual, assigns that individual the stored manifest fitness, associates it with the encoded latent mean, and fills the remaining positions with evaluated decodings of perturbations around that mean. Elitist survival allows the source individual to remain the final raw best. CEM initializes a diagonal normal distribution at the encoded mean, while CMA-ES initializes its mean and covariance-based distribution at the same location. Both distribution methods decode and evaluate their proposed latent vectors, including the current mean; neither inserts the unchanged source string with its stored label into the raw optimizer state. The separately evaluated reconstruction of the source latent mean is also a decoded candidate and must not be confused with the stored source genotype.

The nominal budget is approximately 30,000 population slots per source genotype, in addition to method-specific initialization. This number is not interpreted as an exact count of unique simulator evaluations. Decoding failures, simulator failures, duplicate genotypes, retries, and cached scores affect the number of distinct strings encountered and evaluated. The recorded best-so-far histories use 300 generation or iteration steps when a run completes, and summary rows retain the number of unique scored genotypes together with invalid and duplicate counters where the method explicitly provides them.

Every generated candidate is decoded to normalized **f1** text and assigned its final objective through Framsticks *vertpos*; auxiliary predictions are not substituted for this value. Completed artifacts establish two observed simulation configurations for the single-source benchmark. LSO and the standard baseline use `eval-allcriteria.sim`, `deterministic.sim`, `sample-period-2.sim`, and `only-body.sim`. The mini baseline replaces the first standard-baseline file with `eval-allcriteria-mini.sim` and retains the other three.

The mini baseline replaces the first standard-baseline file with `eval-allcriteria-mini.sim` and retains the other three. The manifest aligns starting genotypes across these conditions, but aggregation does not convert their generated-candidate scores into a controlled paired comparison.

Final LSO aggregation treats each of the fifteen fixed checkpoint–optimizer configurations as a separate group of 150 runs. For cross-optimizer and bucket-conditioned summaries, every row is transformed by the same post-search decision rule: the reported retained score is the larger of the raw optimizer score and the stored source fitness, and retained improvement is their difference from that source fitness. Source-retention counts are reported alongside this outcome. In latent EA, these counts identify rows whose raw best is already the embedded stored-label source. In CEM and CMA-ES, they identify rows for which the derived policy selects the source because the freshly evaluated raw search output does not improve it; an exact tie changes no numeric value but is still a no-improvement retention decision. Bucket-conditioned summaries contain 25 runs per fixed configuration and bucket. The standard baseline is the same-chain comparator for LSO, whereas the mini baseline is summarized separately as a distinct simulation condition.

CoSO is executed as a separate family of direct genotype-space runs because its experimental unit differs from the single-source latent-space benchmark. A CoSO run is an archive-building search over a declared genotype pool rather than an optimization attempt paired with one fixed starting genotype. Each run starts with an empty substitution archive, samples **f1** genotypes from the declared pool, evaluates candidates with the Framsticks *vertpos* criterion, and preserves the archive only within that run. When the currently processed genotype reaches local convergence, its fragments may be inserted into the archive and the search proceeds from another genotype sampled from the same pool. No archive state is shared across independent seeds, buckets, or configurations.

The first CoSO stage is a compatibility-threshold selection experiment. It evaluates spatial thresholds combined together together with directional thresholds presented below:

$$\delta_{\text{spatial}} \in \{0, 1, 2, 3\}, \quad \delta_{\text{direction}} \in \{0, 0.5, 1.0, 1.5\},$$

giving sixteen threshold configurations (under the unit-vector interpretation described in Chapter 3, these directional thresholds correspond approximately to identical direction, 30° difference, 60° difference, and slightly more than a right-angle difference, respectively). The grid ranges from exact or near-exact compatibility to increasingly permissive archive retrieval, what allows the experiment to test whether CoSO benefits from conservative fragment matching or from broader reuse of archived fragments. Each configuration is run with 25 independent seeds, a nominal limit of 30,000 within-constraint evaluations (the value arbitrarily set to mimic the population slots), and the available **f1** genotype dataset as the sampling pool. This stage is used to characterize the sensitivity of the substitution archive to permissive or restrictive compatibility criteria; it is not treated as a paired comparison against individual LSO source-genotype runs.

The bucket-aligned CoSO stage uses the same six initial-fitness ranges as the latent-space benchmark, but the alignment is performed at the bucket level rather than at the individual-source level. The directional compatibility threshold is fixed at the selected value, while the spatial-threshold settings are treated as separate CoSO configurations. For each bucket–configuration pair, CoSO is evaluated with 25 independent seeds and a nominal budget of 30,000 within-constraint evaluations. The archive is reset at the beginning of every run, so differences between runs reflect independent archive construction processes.

CoSO aggregation is therefore performed over independent runs grouped by bucket and compatibility configuration. The main reported endpoint is the run-level improvement in *vertpos* over the best unmodified genotype introduced into that same run and the archive/substitution diag-

nostics defined in Section 4.4. CoSO best-so-far histories are interpreted as trajectories of an archive-based multi-start search under a fixed evaluation budget, not as paired convergence curves from the same source genotypes used by the LSO benchmark.

4.6 Reproducibility and computational setup

Chapter 5

Experimental Results

5.1 Autoencoder reconstruction and validity results

The training logs show substantial differences between the initial sequence models and the later tree- and context-aware variants. Validation reconstruction was measured using deterministic decoding from the latent mean on a run-specific 20% validation split. Each split contained 2,178 examples because the runs used the same dataset size and split fraction, but identical validation membership cannot be assumed. Some run configurations did not specify a split seed, whereas others used a fixed seed. The character VAE and the unmasked grammar VAE obtained no exact validation reconstructions in their recorded runs, although their best average string similarities were 41.23% and 36.65%, respectively. The masked grammar VAE reached 0.09% exact accuracy and 47.60% similarity. The selected TreeVAE checkpoint at epoch 80 reached 10.33% exact accuracy and 69.56% similarity, whereas the selected TransformerVAE checkpoint at epoch 90 reached 2.20% and 55.28%. The later masked TreeVAE and LHS-conditioned variants recorded best logged exact accuracies between 16.57% and 31.59% among the ten checkpoints retained for detailed diagnosis. These cross-run values describe the recorded validation outcomes, subject to the possible difference in split membership.

Table 5.1 reports two different reconstruction evaluations and therefore keeps their denominators separate. “Validation” denotes the best checkpoint-associated value recorded during training on the held-out split. “Local” denotes a later deterministic reconstruction pass over the first 512 encodable dataset records after loading the exported checkpoint. The latter records were not a held-out test set, and this pass produced substantially higher accuracies for several checkpoints. Consequently, local exact accuracy, which reached approximately 90–97% for checkpoints 01–08, is a checkpoint-loading and decoding diagnostic rather than an estimate of out-of-sample reconstruction. Both evaluations use exact string equality; the similarity columns are averages of normalized sequence-matching ratios.

The unconditional generation columns characterize a different task. Validity is the fraction of 100 decoded samples accepted by the configured F1 syntax check, while uniqueness is the fraction of distinct strings among all 100 outputs. Checkpoints 01–08 combined high local reconstruction with only 29–46% generated validity. In contrast, TreeVAE-80 produced 72% valid and 83% unique outputs, and TransformerVAE-90 produced 98% valid and 100% unique outputs despite their weaker reconstruction results. Novelty among outputs classified as valid ranged from 55.00% to 100.00%; it was 97.22% for TreeVAE-80 and 100.00% for TransformerVAE-90. These quantities use different denominators: uniqueness concerns all generated strings, whereas novelty concerns valid generated occurrences.

TABLE 5.1: Reconstruction and unconditional generation diagnostics for the ten selected checkpoints. All numeric diagnostic entries are percentages. Validation uses a run-specific split of 2,178 held-out examples, local reconstruction uses 512 dataset records, and generation uses 100 standard-normal latent samples per checkpoint.

ID	Checkpoint type	Val. exact	Val. sim.	Local exact	Local sim.	Valid	Unique
01	LHS standard	31.59	89.00	96.48	99.53	34.00	57.00
02	LHS, latent 256	30.81	89.57	96.88	99.45	29.00	65.00
03	Masked standard	30.39	88.68	96.88	99.56	32.00	69.00
04	LHS standard, seed 42	28.33	87.56	95.12	99.28	40.00	55.00
05	Masked, slow schedule	28.28	87.41	97.07	99.68	34.00	65.00
06	LHS with depth	27.78	88.23	95.90	99.37	38.00	65.00
07	LHS, slow schedule	25.16	85.88	95.51	99.46	46.00	81.00
08	Masked, reconstruction ceiling	16.57	80.94	90.43	98.98	46.00	95.00
09	TreeVAE, epoch 80	10.33	69.56	78.13	96.52	72.00	83.00
10	TransformerVAE, epoch 90	2.20	55.28	19.14	82.28	98.00	100.00

Fresh Framsticks evaluation further showed that syntactic acceptance was not equivalent to useful phenotype generation. For the TreeVAE-80 and TransformerVAE-90 random samples, the mean evaluated *vertpos* values were 0.186 and 0.158, with maxima of 0.852 and 0.739. Most random outputs from several high-reconstruction LHS or masked checkpoints either could not be assigned a numeric value or remained near the minimal observed value. These 100-sample diagnostics were used to describe generator behaviour, not to rank optimization methods.

The fifth checkpoint later included in the final optimization benchmark was a fitness-aware LHS model and was not part of the ten-checkpoint table. Its exported best-reconstruction checkpoint recorded 398 exact validation reconstructions out of 2,178 (18.27%) and 83.81% average similarity at epoch 440. At the same validation point, its auxiliary normalized-fitness head recorded an MSE of 0.0834 and a correlation of 0.9576. These are auxiliary representation-learning diagnostics; the optimization results below continue to use true Framsticks *vertpos* rather than the prediction head.

5.2 CoSO optimization results

The CoSO experiments produced two complementary result sets. The first one was a compatibility-threshold screening stage with sixteen threshold pairs and 25 independent runs per pair. The second one was the bucket-aligned 30,000-evaluation experiment, in which the directional threshold was fixed at 0.5 and spatial thresholds 0, 1, 2, and 3 were evaluated separately in each of the six initial-fitness buckets.

Table 5.2 reports the mean final best *vertpos* values and their run-to-run variability from the threshold-screening stage. The highest mean values were obtained for spatial threshold 0, with small differences between directional thresholds. However, this stage used the broader genotype dataset as the sampling pool, so final best scores were strongly affected by the best genotypes introduced during the multi-start search. The mean improvement over the best introduced genotype remained small in this screening stage. Therefore, the threshold-screening results are interpreted mainly as sensitivity evidence for archive retrieval, not as the final CoSO optimization result.

TABLE 5.2: Mean final best *vertpos* in the CoSO threshold-screening stage. Each cell summarizes 25 independent runs with a nominal limit of 30,000 within-constraint evaluations.

Spatial threshold	Direction 0	Direction 0.5	Direction 1.0	Direction 1.5
0	2.103 $\pm$ 0.083	2.108 $\pm$ 0.116	2.102 $\pm$ 0.117	2.112 $\pm$ 0.102
1	2.078 $\pm$ 0.119	2.038 $\pm$ 0.099	2.049 $\pm$ 0.137	2.034 $\pm$ 0.076
2	2.033 $\pm$ 0.143	2.055 $\pm$ 0.095	2.036 $\pm$ 0.087	2.068 $\pm$ 0.128
3	2.031 $\pm$ 0.135	2.026 $\pm$ 0.127	1.958 $\pm$ 0.089	2.027 $\pm$ 0.111

Figure 5.1 complements Table 5.2 by showing how the mean best-so-far value develops during the threshold-screening runs. The endpoint ranking is similar across some threshold pairs, but the

trajectories show that most improvement occurs early and that differences between configurations are relatively small after the initial phase.

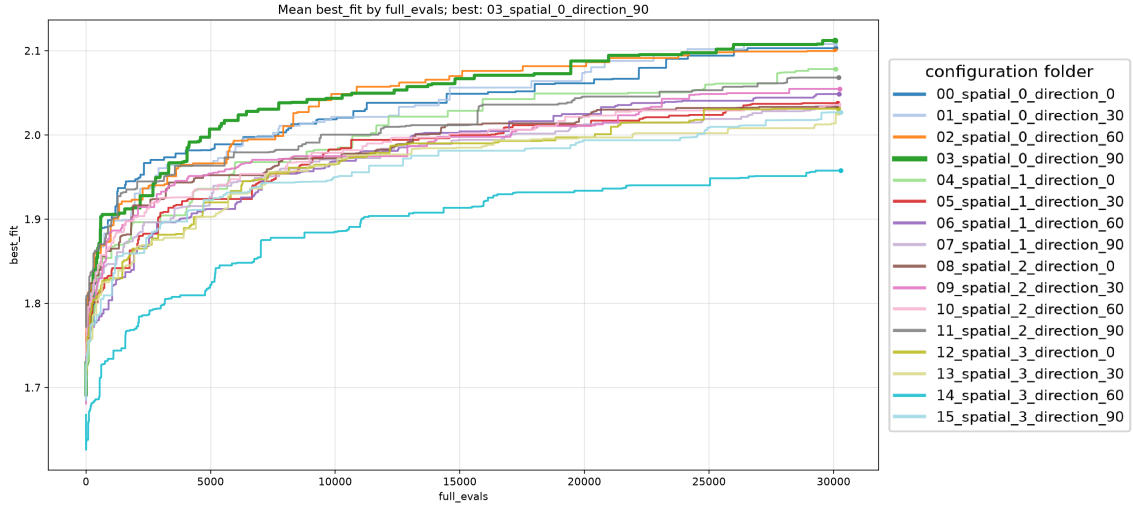


FIGURE 5.1: Mean CoSO best-so-far trajectories in the compatibility-threshold screening stage. Each line averages 25 independent runs for one spatial-directional threshold pair. The trajectories describe archive-based multi-start search progress, not paired convergence from fixed source genotypes.

The bucket-aligned stage provides the main CoSO optimization evidence. Table 5.3 aggregates the 150 runs available for each spatial threshold across all six buckets. Spatial thresholds 0 and 1 produced the strongest results. Their mean final best values were 1.621 and 1.611, respectively, and their mean run-level substitution improvements were 0.465 and 0.455. Spatial threshold 0 also produced the largest single observed CoSO result, 2.270. In contrast, spatial threshold 3 produced the weakest aggregate outcome, with mean final best value 1.191 and mean run-level substitution improvement 0.038.

TABLE 5.3: Bucket-aligned CoSO results by spatial threshold. Directional threshold is fixed at 0.5. Each row summarizes 150 runs, with 25 runs in each of six buckets. $\Delta_{\text{subst}}^{\text{CoSO}}$ is the final best value minus the best unmodified genotype introduced in the same run.

Spatial threshold	Mean best	Median best	SD	Maximum	Mean introduced best	Mean $\Delta_{\text{subst}}^{\text{CoSO}}$
0	1.621	1.581	0.305	2.270	1.156	0.465
1	1.611	1.622	0.315	2.378	1.156	0.455
2	1.536	1.528	0.376	2.296	1.156	0.381
3	1.193	1.127	0.458	2.011	1.156	0.038

The bucket-conditioned improvements in Table 5.4 show that CoSO improved weakest and medium-quality genotype pools most clearly, especially for spatial thresholds 0 and 1. For these two thresholds, the mean run-level substitution improvement exceeded 0.77 in B1, remained above 0.62 in B2, and stayed around 0.42–0.45 in B4. Spatial threshold 2 produced a weaker but still consistent improvement pattern across all buckets, with values from 0.579 in B1 to 0.171 in B6. In contrast, spatial threshold 3 was largely ineffective under this metric, with only 0.148 improvement in B1 and near-zero or zero improvement in the remaining buckets. The strongest bucket B6 showed much smaller gains for the best-performing spatial thresholds, between 0.107 and 0.171. This pattern means that CoSO was most useful when the starting pool still left substantial room for improvement. The decline in improvement for B6 should not be read as poor final quality: B6

starts from already strong genotypes, so high final scores can coincide with smaller margins over the best introduced genotype.

TABLE 5.4: Mean run-level CoSO substitution improvement by bucket. Values report $F_{\text{CoSO},r}^{\text{best}} - F_r^{\text{introduced}}$, not improvement over a single paired source genotype.

Spatial threshold	B1	B2	B3	B4	B5	B6
0	0.828	0.629	0.427	0.425	0.370	0.113
1	0.779	0.626	0.407	0.447	0.365	0.107
2	0.579	0.504	0.401	0.356	0.274	0.171
3	0.148	0.037	0.027	0.015	0.000	0.000

The archive diagnostics support the same interpretation. The least effective spatial threshold produced many local convergence events and a large archive, but this did not translate into meaningful improvement beyond the best introduced genotype. More restrictive spatial thresholds produced higher final quality and larger substitution gains, indicating that tighter spatial compatibility made archived fragments more useful as replacements. The results therefore favour spatial thresholds 0 and 1 under the 30,000-evaluation bucket-aligned protocol.

These results are interpreted as within-CoSO evidence. CoSO runs are archive-based multi-start searches over genotype pools, so their best-so-far histories and final scores are not paired per-source outcomes in the same sense as LSO runs. The CoSO results can be compared with LSO only at the aggregate bucket or configuration level under the common nominal budget, not as one-to-one convergence curves from identical starting genotypes.

5.3 Latent-space optimization results

The final Latent Space Optimization benchmark produced 2,250 completed runs: 150 starting genotypes for each of five checkpoints and three optimizers. Every configuration therefore contains the same 25 starting genotypes from each of the six initial-fitness buckets. The raw best-score field is not directly comparable across the three optimizers: latent EA can return the original genotype with its stored manifest fitness, whereas CEM and CMA-ES return freshly evaluated decoded search outputs without that fallback. Table 5.5 therefore applies the uniform retained-source decision policy from Section 4.4. For every row, the retained score is the larger of the raw result and the stored source fitness. Each cell reports the mean, sample standard deviation, and maximum over the 150 retained decision scores of one fixed configuration; it is not a generated-candidate-only measure and does not select an optimizer separately for each source genotype.

TABLE 5.5: Final LSO results under the uniform source-retention policy. Cells show mean $\pm$ sample standard deviation [maximum] of retained *vertpos*, defined as the larger of the raw optimizer output and the stored source fitness, with $n = 150$ runs per cell.

Checkpoint	Latent-space EA	CEM	CMA-ES
TreeVAE-80	1.772 $\pm$ 0.228 [2.331]	1.179 $\pm$ 0.354 [2.142]	1.638 $\pm$ 0.231 [2.353]
TransformerVAE-90	1.774 $\pm$ 0.199 [2.129]	1.213 $\pm$ 0.304 [2.129]	1.506 $\pm$ 0.226 [2.129]
LHS, latent 256	1.477 $\pm$ 0.203 [2.129]	1.441 $\pm$ 0.150 [2.129]	1.416 $\pm$ 0.156 [2.129]
LHS with depth	1.439 $\pm$ 0.210 [2.129]	1.430 $\pm$ 0.166 [2.129]	1.448 $\pm$ 0.141 [2.129]
Fitness-aware LHS	1.612 $\pm$ 0.242 [2.201]	1.481 $\pm$ 0.190 [2.153]	1.619 $\pm$ 0.296 [2.264]

The two largest retained-score means were obtained by latent EA with TransformerVAE-90 and TreeVAE-80, at 1.774 and 1.772; their medians were 1.843 and 1.755, and their mean retained improvements were 0.764 and 0.763. Latent EA also had the largest mean for the latent-256 LHS checkpoint. The ranking changed for the other two representations: CMA-ES had the largest

retained mean for LHS with depth, at 1.448 compared with 1.439 for latent EA, and for fitness-aware LHS, at 1.619 compared with 1.612. The search algorithm alone therefore did not determine the retained decision outcome; the decoded representation remained an important experimental factor.

CEM had the lowest optimizer-wide retained mean and was particularly weak with TreeVAE-80 and TransformerVAE-90, where its retained means were 1.179 and 1.213. It was more competitive with the LHS-based checkpoints, reaching retained means between 1.430 and 1.481, and it exceeded CMA-ES for the latent-256 LHS model. CMA-ES reached the largest single LSO score, 2.353 with TreeVAE-80, but this high maximum did not translate into the largest mean for that checkpoint. Its retained checkpoint means varied from 1.416 to 1.638, and the fitness-aware combination had the largest within-configuration standard deviation in Table 5.5, 0.296. Thus, isolated high results from CMA-ES should not be interpreted as uniform performance.

Source retention occurred in 83 of 750 latent-EA rows. In each of these rows, the raw result already identified the original genotype and its stored label as best; 62 of the 83 cases were in B6. Under the uniform decision policy, the source was retained in 132 of 750 CEM rows and 85 of 750 CMA-ES rows. For CEM, 131 cases required a strict post-hoc replacement of a worse freshly evaluated raw output and one case was an exact no-improvement tie; all 85 CMA-ES cases required replacement. The B6 counts were 82 for CEM and 65 for CMA-ES. These counts quantify the decision policy and the protocol asymmetry, not the number of failures to generate candidates.

The retained-score ranking also reinforces the distinction between reconstruction and optimization quality. The latent-256 LHS and LHS-depth checkpoints had logged validation exact accuracies of 30.81% and 27.78%, yet their best retained means across optimizers were 1.477 and 1.448. TransformerVAE-90 had only 2.20% validation exact reconstruction but achieved the largest retained mean when paired with latent EA. TreeVAE-80 showed a similar pattern, combining moderate reconstruction with strong latent-EA and CMA-ES outcomes. Reconstruction quality was therefore informative about one property of a checkpoint, but it was not a sufficient selection criterion for latent search.

The results in Table 5.5 are therefore first interpreted as within-LSO outcomes. The completed LSO details files record the same four-file chain as the standard Framsticks EA baseline: `eval-allcriteria.sim`, `deterministic.sim`, `sample-period-2.sim`, and `only-body.sim`. The shared 150 starting genotypes therefore permit same-chain alignment with the standard baseline; the separate baseline comparison is discussed in following Section 5.4.

5.4 Comparison with the Framsticks evolutionary baseline

The completed LSO outputs and the standard Framsticks EA baseline constitute a same-chain comparison condition. Both score generated candidates with `eval-allcriteria.sim`, `deterministic.sim`, `sample-period-2.sim`, and `only-body.sim`, and both use the same 150 starting genotypes. The second baseline uses the corresponding four-file chain beginning with `eval-allcriteria-mini.sim`. Consequently, Table 5.6 reports the standard baseline as the same-chain reference for LSO and the mini baseline as a separate descriptive condition.

TABLE 5.6: Standalone Framsticks EA results under the two completed baseline simulation configurations. Values summarize 150 runs per condition; SD is the sample standard deviation, and mean improvement is relative to the stored source-genotype fitness.

Baseline condition	Mean	Median	SD	Maximum	Mean improvement
Standard four-file chain	1.690	1.669	0.343	2.368	0.680
Mini four-file chain	1.693	1.656	0.364	2.653	0.683

The relation between CoSO and the Framsticks EA baseline is different from the relation between LSO and the baseline. CoSO evaluates candidates with the same four-file chain as the mini baseline, beginning with `eval-allcriteria-mini.sim`. Therefore, the mini baseline is the closest available reference condition for CoSO. However, the experimental unit is still different: the mini baseline contains one evolutionary run for each manifest genotype, whereas a CoSO run is an archive-based multi-start search over a bucket-level genotype pool. Consequently, ordinary paired per-source differences and win rates are not reported for CoSO either.

At the aggregate level, the strongest bucket-aligned CoSO configuration reached a mean final best *vertpos* of 1.529 under the 30,000-evaluation protocol, with the largest observed CoSO value equal to 2.384. The mini Framsticks EA baseline reached a standalone mean of 1.693 and maximum of 2.653. These values indicate that the completed mini-baseline run remained stronger as a standalone optimizer under its recorded protocol. CoSO should therefore be interpreted primarily through its substitution-specific evidence, bucket-conditioned improvements, and archive behaviour, rather than as outperforming the Framsticks EA baseline in the available experiments.

Both baseline datasets contain 150 final rows, one for each manifest genotype, and every history contains 300 generations. Each run recorded 30,100 unique entries in its score cache, including the stored source entry and initialization. Under the standard condition, the mean best *vertpos* was 1.690 and the largest observed value was 2.368. Under the mini condition, the corresponding standalone values were 1.693 and 2.653. These values characterize separate simulator configurations; the larger mini-condition maximum is not evidence that one baseline variant is superior under a common objective.

What can be concluded is explicit but still bounded by the reported aggregates. The standard Framsticks EA execution provides the same-chain baseline distribution for the LSO runs, while the mini execution remains a separately labelled simulation condition. Section 5.3 identifies the strongest fixed LSO configurations within the completed design space. Stronger claims about per-seed superiority should use the aligned standard-baseline rows and the paired metrics defined in Section 4.4.

5.5 Comparison between CoSO and Latent Space Optimization

The completed experiments do not support a strict paired comparison between CoSO and LSO. LSO runs are organized around individual starting genotypes, whereas CoSO runs are archive-based multi-start searches over bucket-level genotype pools. Therefore, the comparison below uses aggregate bucket-conditioned trends rather than per-source win rates. For LSO, improvement is measured as retained improvement over the stored source genotype. For CoSO, improvement is measured as the run-level gain over the best unmodified genotype introduced into the same run.

Table 5.7 compares the two strongest LSO configurations from Table 5.8 with the strongest CoSO spatial-threshold settings from Table 5.4. The values should be read as method-specific

improvement margins, not as paired differences. They answer whether each method improved beyond its own reference point within the same initial-fitness bucket.

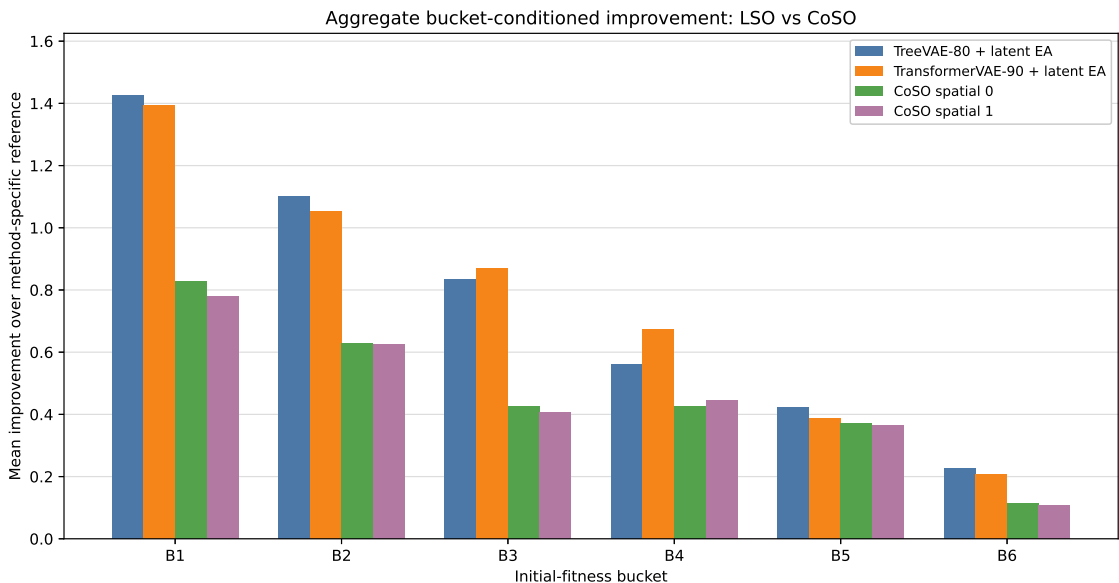
TABLE 5.7: Aggregate bucket-conditioned improvement for selected LSO and CoSO configurations. LSO values are retained improvements over the stored source genotype. CoSO values are run-level improvements over the best unmodified genotype introduced in the same run. The comparison is descriptive and bucket-level, not paired per source genotype.

Configuration	B1	B2	B3	B4	B5	B6
TreeVAE-80 + latent EA	1.426	1.103	0.835	0.562	0.422	0.227
TransformerVAE-90 + latent EA	1.394	1.054	0.869	0.673	0.388	0.206
CoSO spatial 0	0.828	0.629	0.427	0.425	0.370	0.113
CoSO spatial 1	0.779	0.626	0.407	0.447	0.365	0.107

The selected LSO configurations show larger mean improvement margins in all buckets. The difference is especially visible in the weakest buckets: in B1, the two leading LSO configurations reached improvements of 1.426 and 1.394, whereas the two strongest CoSO configurations reached 0.828 and 0.779. The same pattern remains in B2 and B3. In the medium and stronger buckets, the gap becomes smaller, but LSO still retains higher aggregate improvement values.

The comparison also shows a shared qualitative trend. Both approaches produce their largest measurable gains in the weakest bucket and smaller gains in the strongest bucket. This supports the interpretation that weak starting material leaves more room for optimization, whereas high-fitness starting material can produce high final scores with smaller improvement margins. The trend is visible despite the different search mechanisms: LSO explores a learned continuous representation around one encoded source genotype, while CoSO reuses compatible fragments through an archive accumulated during multi-start genotype-space search.

The result should not be interpreted as a direct proof that one method dominates the other on paired starting genotypes. The LSO values include a source-retention decision policy, while CoSO values use the best genotype introduced into the run as their reference. Moreover, CoSO trajectories summarize archive-building progress over a pool rather than convergence from a single source genotype. The comparison therefore supports an aggregate conclusion: within the completed experiments, the strongest LSO configurations achieved larger bucket-conditioned improvement margins, while CoSO provided interpretable genotype-space improvements without relying on a trained latent representation.



5.6 Analysis by initial fitness ranges

This section summarizes how the observed optimization behaviour changes across the initial-fitness buckets. For LSO, the analysis uses the retained-source policy because runs are paired with individual starting genotypes. Every LSO cell contains the same 25 source genotypes from one predefined bucket and one fixed checkpoint–optimizer configuration. Figure 5.2 reports mean retained improvement, $F^{\text{retained}} - F^{\text{source}}$, for all fifteen configurations. It does not measure generated-candidate-only optimizer performance, select the best configuration separately for each source genotype, or contain Framsticks EA values. For CoSO, the corresponding bucket analysis uses run-level pool summaries from Section 5.2, because a CoSO run is an archive-based multi-start search rather than a single-source optimization run. The two analyses therefore share the same bucket definitions, but not the same experimental unit.

All fifteen fixed configurations have a smaller mean retained improvement in B6 than in B1, although the intermediate buckets do not form a strictly monotonic sequence in every case. This endpoint decline is not equivalent to a decline in the retained final score. The mean stored source fitness increases from 0.411 in B1 to 1.688 in B6, so a high-fitness bucket can have a larger retained score but a smaller margin over its starting labels. Table 5.8 illustrates this distinction for the two configurations that had the largest overall retained means in Section 5.3; they are fixed across every bucket.

TABLE 5.8: Bucket-conditioned means for the two globally leading fixed LSO configurations under the source-retention policy. Each bucket contains 25 runs. “Retained” is mean retained *vertpos*; Δ is mean retained improvement from the stored source fitness.

Bucket	Source mean	TreeVAE-80 retained	Tree Δ	TransformerVAE-90 retained	Transformer Δ
B1	0.411	1.836	1.426	1.805	1.394
B2	0.640	1.743	1.103	1.694	1.054
B3	0.897	1.733	0.835	1.766	0.869
B4	1.109	1.671	0.562	1.783	0.673
B5	1.313	1.736	0.422	1.702	0.388
B6	1.688	1.915	0.227	1.894	0.206

The TreeVAE checkpoint at epoch 80 with latent EA had mean retained improvements of 1.426 in B1 and 0.227 in B6. For the TransformerVAE checkpoint at epoch 90, the corresponding values decreased from 1.394 to 0.206. Their mean retained scores nevertheless remained high in B6, at 1.915 and 1.894, because the source genotypes in that bucket were already strong. The trend therefore supports a diminishing retained-improvement interpretation rather than a claim that latent search produces its lowest absolute outcomes from elite starting points.

The same endpoint pattern is visible outside the two leading configurations, but its magnitude depends on both checkpoint and optimizer. The retained policy eliminates negative improvement by construction, so the earlier negative raw differences for CEM and CMA-ES are not interpreted as comparative outcomes. Instead, stronger buckets are characterized by more frequent source retention. Across all five checkpoints, latent EA retained the embedded source in 62 of 125 B6 rows. The policy retained the source in 82 of 125 CEM rows and 65 of 125 CMA-ES rows in B6, compared with no retained sources for any optimizer in B1 or B2. The B6 heatmap cells consequently combine successful generated improvements with zero-improvement source-retention decisions. This concentration shows that strong source genotypes more often remain preferable under the stated policy; it does not recover the unavailable generated-candidate-only latent-EA scores.

These results concern only the retained-source decision policy across starting-fitness ranges

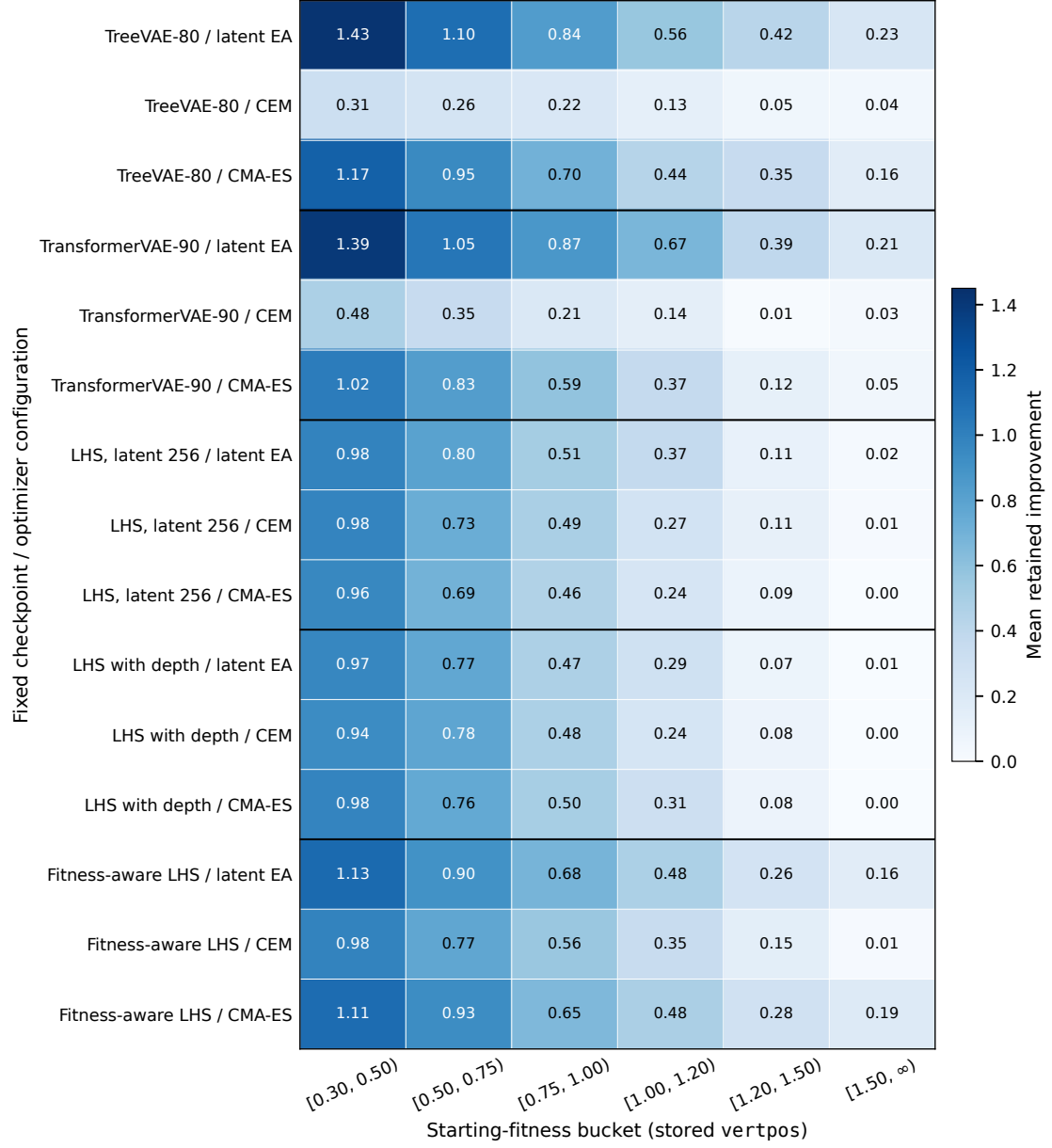


FIGURE 5.2: Bucket-conditioned mean retained improvement for each fixed LSO checkpoint–optimizer configuration. Each cell contains 25 runs from the completed one-file LSO condition (`eval-allcriteria.sim`) and reports $\max(F^{\text{raw}}, F^{\text{source}}) - F^{\text{source}}$. Zero means that the source is retained. Values are decision-policy outcomes, not generated-candidate-only performance; no per-seed selection is used.

inside the standard four-file LSO condition. They are not themselves bucket-conditioned win rates against the standard baseline; such quantities require explicit paired aggregation with the aligned baseline rows. They also should not be converted into comparisons with the mini baseline, because the mini baseline uses a different first simulation file. The stored source labels define the retention rule and provide a common within-LSO reference.

The CoSO bucket-aligned results show a related but not identical dependence on initial fitness. The largest run-level substitution improvements were observed in the weakest buckets, especially for spatial thresholds 0 and 1, where B1 improvements reached 0.828 and 0.779, respectively. Improvements decreased in the strongest bucket, where the same thresholds reached only 0.113 and 0.107. Spatial threshold 2 followed the same general trend but with smaller gains in the weaker buckets, while spatial threshold 3 produced little or no improvement beyond the best introduced genotype in most buckets. This mirrors the LSO retained-improvement pattern at a qualitative level: weak starting material leaves more room for measurable improvement, whereas strong starting material can yield high final scores but smaller margins over the reference genotype or pool.

This similarity is qualitative rather than paired. Again, LSO bucket results summarize fixed source-genotype runs under the retained-source policy, whereas CoSO bucket results summarize archive-based runs over bucket-level pools. Therefore, the bucket trends can be compared as aggregate sensitivity patterns, but not as per-source method differences.

5.7 Summary of results

The autoencoder experiments show that reconstruction, unconditional generation, and optimization measure different properties. Several LHS and masked checkpoints reached substantially higher validation reconstruction than TreeVAE-80 and TransformerVAE-90, while the latter produced markedly higher unconditional validity and uniqueness. The subsequent search results confirm that validation reconstruction alone is not a sufficient checkpoint-selection criterion for Latent Space Optimization.

The final LSO condition contains 2,250 runs covering fifteen fixed checkpoint-optimizer configurations and 150 common starting genotypes. Comparative summaries use the uniform retained score rather than the method-dependent raw best score. TransformerVAE-90 and TreeVAE-80 with latent EA had the two largest retained means, 1.774 and 1.772. Latent EA did not lead every checkpoint: CMA-ES had the largest retained mean for LHS with depth and fitness-aware LHS. CEM had the lowest optimizer-wide retained mean, whereas CMA-ES produced high isolated maxima, including the largest LSO score of 2.353, but with lower means or greater dispersion than the two strongest latent-EA configurations. These findings identify effective fixed combinations within the investigated LSO design space; they do not identify a universally best autoencoder or optimizer.

The CoSO experiments showed that the archive-based substitution mechanism was sensitive to the spatial compatibility threshold. In the threshold-screening stage, several configurations reached similar final best values, but these full-dataset runs were strongly affected by the best genotypes introduced during multi-start search. The bucket-aligned results were therefore more informative about substitution behaviour. In that setting, spatial thresholds 0 and 1 produced the strongest improvements, especially in the weakest and medium-quality buckets, whereas spatial threshold 3 produced little or no improvement beyond the best introduced genotype in most buckets.

Across the six buckets, every fixed configuration had lower mean retained improvement in the strongest source bucket than in the weakest. The intermediate buckets need not be monotonic.

Source retention was concentrated in the upper buckets: latent EA already embedded the fallback in its raw output, while CEM and CMA-ES received it through the derived policy. The result supports the interpretation that the policy more often selects generated improvements for weak and medium source genotypes, whereas already strong sources are retained more frequently. It is not evidence about generated-candidate-only latent-EA performance.

The same qualitative dependence on initial fitness was visible for CoSO, but with a different experimental unit. CoSO improvements were largest for weak starting pools and smaller for the strongest bucket, where the available genotypes already had high stored fitness values. This supports the general conclusion that both representation-aware approaches show larger measurable gains when the initial material leaves more room for improvement, although LSO and CoSO bucket results remain aggregate trends rather than paired per-source comparisons.

The standard Framsticks EA baseline uses the same 150 starting-genotype manifest and the same four-file simulation chain as LSO. The mini baseline remains a separate simulation condition because it begins with `eval-allcriteria-mini.sim` instead of `eval-allcriteria.sim`. CoSO is closer to the mini-baseline simulation chain, but its archive-based multi-start run structure still prevents ordinary paired per-source comparison.

Chapter 6

Discussion and Conclusions

6.1 Strengths and limitations of Compatible Substitutions Optimization

The main strength of CoSO is that it performs representation-aware search directly in the original genotype space. Unlike latent-space methods, it does not require training a generative model, selecting checkpoints, or decoding continuous vectors back into genotype strings. Candidate changes are expressed as explicit fragment substitutions in **f1**, and every accepted modification can be inspected as a replacement of one genotype fragment by another. This makes the method comparatively transparent: the role of the archive, compatibility thresholds, accepted substitutions, and local convergence events can be analysed directly from the run logs.

A second strength is that CoSO can reuse information gathered during search. Fragments extracted from locally converged genotypes are stored in the archive and can later become candidate replacements in different genotype contexts. The bucket-aligned results indicate that this mechanism was most useful when the starting genotype pool left substantial room for improvement. In the weaker and medium-fitness buckets, the best CoSO configurations produced clear improvements beyond the best introduced genotype. This suggests that explicit compatible substitutions can provide useful search moves without relying on a learned continuous representation.

The method also offers interpretable control through its compatibility thresholds. The spatial one determines how similar the geometric displacement of two fragments must be, while the directional threshold limits differences between their terminal directions. The experiments showed that this control is important: overly weak or poorly matched spatial compatibility produced much smaller improvements, whereas the stronger configurations benefited from more effective fragment reuse. CoSO therefore exposes a meaningful trade-off between conservative fragment matching and broader archive retrieval.

The most important limitation is that CoSO depends on hand-designed representation features. In this thesis, compatibility is defined using spatial and directional descriptors derived from Framsticks **f1** fragments. These descriptors are meaningful for the studied representation, but they are not automatically transferable to arbitrary genotype languages or tasks. Applying CoSO to a different domain would require designing new fragment extraction rules, feature descriptors, and compatibility thresholds.

A second limitation follows from the archive-based multi-start structure of the algorithm. A CoSO run is not a single-source optimization trajectory. It may process several genotypes from the input pool while retaining an archive accumulated from previous locally converged solutions. This makes CoSO suitable for bucket-level and configuration-level analysis, but it prevents ordinary

paired per-source comparisons with methods such as LSO or a standard evolutionary baseline. The best-so-far histories should therefore be interpreted as progress curves of an archive-building search, not as convergence curves from one fixed source genotype.

The current implementation is also greedy at the level of accepted substitutions. A candidate replacement is retained only when it improves the current genotype according to the evaluated objective. This makes the search behaviour simple and easy to interpret, but it can also limit exploration. Potentially useful neutral or temporarily harmful substitutions are discarded, even if they could enable later improvements. As a result, CoSO may converge locally and repeatedly restart rather than traverse low-fitness intermediate regions.

Finally, the experimental results show that building a large archive is not sufficient by itself. Some configurations produced many archive entries or convergence events without producing strong final improvements. The usefulness of the archive depends on whether retrieved fragments are compatible in a way that leads to better complete genotypes. CoSO is therefore best understood as a promising, interpretable genotype-space mechanism whose effectiveness depends on the quality of the fragment descriptors, the chosen compatibility thresholds, and the structure of the available genotype pool.

6.2 Strengths and limitations of Latent Space Optimization

The principal strength of Latent Space Optimization is the separation between representation learning and search. A trained autoencoder converts a variable-length **f1** genotype into a fixed-dimensional vector, after which different derivative-free optimizers can be applied without changing the final objective. The latent-space EA, CEM, and CMA-ES all used the same decode-validate-evaluate interface and received true Framsticks *vertpos* feedback for generated candidates, although their handling of the stored source differed as discussed below. This decoupling made it possible to compare several search strategies on the same learned representation and, conversely, to reuse one search strategy across several checkpoint families.

Grammar-based representation is another important strength. Production sequences, derivation context, and grammar masks provide explicit structure that is unavailable to an unconstrained character decoder. They make syntactic admissibility testable and can restrict locally incompatible production choices before simulator evaluation. The benefit is nevertheless conditional rather than absolute. In the unconditional checkpoint diagnostic, generated validity ranged from 29% to 98%, even though all ten selected checkpoints represented **f1** genotypes through grammar-related models. Grammar awareness therefore reduced the representation problem, but did not guarantee a complete admissible derivation, a simulatable organism, or high fitness at every latent point.

Under the uniform source-retention decision policy, the largest within-LSO means came from latent EA combined with TreeVAE-80 or TransformerVAE-90. Their mean retained scores were 1.772 and 1.774, the two largest means among the fifteen evaluated checkpoint-optimizer configurations. This result identifies effective fixed representation-search combinations within the investigated LSO design space; it is not generated-candidate-only performance and does not by itself establish an advantage over the Framsticks EA baseline without the paired statistics defined earlier. The completed LSO runs and the standard baseline runs used the same four-file simulation chain, so the remaining caution concerns retained-policy semantics and checkpoint dependence rather than a simulation-chain mismatch.

This result identifies effective fixed representation-search combinations within the investigated LSO design space; it is not generated-candidate-only performance and, by itself, is not a baseline comparison. The completed LSO runs and the standard Framsticks EA baseline use the same

four-file simulation chain and the same starting-genotype manifest, so LSO can be compared with this standard baseline at the paired source-genotype level. This does not apply to the mini baseline or to CoSO: the mini baseline uses a different first simulation file, while CoSO has an archive-based multi-start experimental unit, so direct comparison with LSO will be conducted in different way.

A central limitation is checkpoint dependence. Across the five checkpoints, the latent-space EA retained mean ranged from 1.439 to 1.774. CEM and CMA-ES also changed substantially with the representation. CEM had the lowest optimizer-wide retained mean and was particularly weak with TreeVAE-80 and TransformerVAE-90, although it was more competitive for some LHS checkpoints. CMA-ES reached high maxima, including the largest LSO result of 2.353, and had the largest retained mean for the LHS-depth and fitness-aware checkpoints; these outcomes were nevertheless accompanied by lower central performance than the two leading latent-EA configurations or substantial dispersion in some configurations. Selecting an optimizer without jointly validating the checkpoint is therefore unlikely to be reliable.

The reconstruction results explain why checkpoint selection cannot rely on autoencoder accuracy alone. The LHS latent-256 checkpoint reconstructed 30.81% of its validation examples exactly, whereas TransformerVAE-90 reconstructed only 2.20%. Nevertheless, the latter had a markedly larger retained mean with latent EA. Exact reconstruction rewards faithful inversion near observed examples, while optimization also depends on neighbourhood smoothness, the diversity of decoded perturbations, and whether movement in latent space reaches useful phenotypes. The experiments measure these properties only indirectly, but they clearly show that good reconstruction is not sufficient for a strong retained optimization outcome.

The completed protocol also has a source-retention asymmetry. Latent EA inserts the unchanged genotype with its stored manifest fitness and can return that stored-label individual as its raw best; this occurred in 83 of 750 runs. CEM and CMA-ES instead return freshly evaluated decoded candidates and have no equivalent raw fallback. Applying $\max(F^{\text{raw}}, F^{\text{source}})$ creates a uniform decision policy and changes the leading optimizer for the LHS-depth and fitness-aware checkpoints, but it cannot reconstruct generated-candidate-only values for the latent-EA runs in which the source remained best. The retained metric is therefore appropriate for the practical decision to keep or replace the source, while raw cross-optimizer rankings from the completed protocol are not methodologically uniform. A future experiment should evaluate the source through the same chain for every optimizer and record generated-candidate-only and retained outcomes separately.

Validity and duplication create an additional search cost. The unconditional generation results ranged from 55% to 100% unique strings, and high reconstruction did not imply high random-generation validity. The latent-space EA outputs also recorded large checkpoint-dependent retry counters. Across 300-generation runs, the median invalid-candidate count ranged from 2,137 for TransformerVAE-90 to 92,624 for the latent-256 LHS checkpoint; median duplicate counts ranged from 9 to 19,390. These counters are available for the latent-space EA only and should not be treated as directly comparable CEM or CMA-ES measurements. They nevertheless show that some decoders required many proposals to obtain the requested accepted offspring, reducing the practical efficiency of the learned representation.

LSO also retains the main computational cost of black-box evolutionary design. Neural encoding and decoding do not replace phenotype evaluation: candidate fitness is still obtained from Framsticks. With populations of 100 and 300 generations or iterations, a single run has approximately 30,000 nominal candidate proposals, or evaluation slots. This nominal budget is not the number of unique simulator evaluations: invalid candidates, repeated genotypes, retries, and cached values make both the proposal count and the number of unique evaluated genotypes

method-dependent. The autoencoder adds training, checkpoint storage, and inference overhead on top of this simulator cost. Its value lies in proposing a useful search geometry, not in removing the expensive objective evaluation.

Finally, the evidence supports only limited generalization. The final LSO benchmark used five checkpoints, three optimizers, one **f1** dataset, one fitness criterion, the standard four-file simulation chain, and 150 structured starting genotypes. Results may change for another genotype language, simulator configuration, fitness objective, training distribution, or latent dimensionality. The standard baseline shares this chain and the starting-genotype manifest, so it is the same-condition baseline for LSO. The mini baseline remains a separate condition because it uses the corresponding four-file chain beginning with `eval-allcriteria-mini.sim`. Broader conclusions require repeated training seeds, independently selected checkpoints, additional tasks, and evaluation conditions that remain identical across the methods being compared.

6.3 Comparative discussion

6.4 Threats to validity

6.5 Conclusions

6.6 Future work

Bibliography

- [1] Peter Bentley. An introduction to evolutionary design by computers. *Evolutionary design by computers*, pages 1–73, 1999.
- [2] Peter J Bentley. Three ways to grow designs: A comparison of embryogenies for an evolutionary design problem. 1999.
- [3] Peter J. Bentley, Soo Ling Lim, Adam Gaier, and Linh Tran. Evolving through the looking glass: Learning improved search spaces with variational autoencoders. In *Parallel Problem Solving from Nature – PPSN XVII*, volume 13398 of *Lecture Notes in Computer Science*, pages 371–384. Springer, 2022.
- [4] Peter A. N. Bosman and Dirk Thierens. Linkage neighbors, optimal mixing and forced improvements in genetic algorithms. In *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO ’12, pages 585–592, New York, NY, USA, 2012. ACM.
- [5] Rafael Gómez-Bombarelli, Jennifer N. Wei, David Duvenaud, José Miguel Hernández-Lobato, Benjamín Sánchez-Lengeling, Dennis Sheberla, Jorge Aguilera-Iparraguirre, Timothy D. Hirzel, Ryan P. Adams, and Alán Aspuru-Guzik. Automatic chemical design using a data-driven continuous representation of molecules. *ACS Central Science*, 4(2):268–276, 2018.
- [6] Nikolaus Hansen. The CMA evolution strategy: A tutorial, 2016.
- [7] Geoffrey E. Hinton and Ruslan R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006.
- [8] Gregory S Hornby, Hod Lipson, and Jordan B Pollack. Generative representations for the automated design of modular physical robots. *IEEE transactions on Robotics and Automation*, 19(4):703–719, 2003.
- [9] Piotr Kaszuba, Maciej Komosiński, and Agnieszka Mensfelt. Automated development of latent representations for optimization of sequences using autoencoders. In *2021 IEEE Congress on Evolutionary Computation (CEC)*, pages 1360–1367, 2021.
- [10] Rafal Kicinger, Tomasz Arciszewski, and Kenneth De Jong. Evolutionary computation and structural design: A survey of the state-of-the-art. *Computers & structures*, 83(23-24):1943–1978, 2005.
- [11] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *Proceedings of the International Conference on Learning Representations*, 2014.
- [12] Maciej Komosiński and Konrad Miazga. GOM-based compatible substitutions optimization for variable-length representation gray-box problems. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, GECCO ’25 Companion, pages 571–574, New York, NY, USA, 2025. ACM.
- [13] Matt J. Kusner, Brooks Paige, and José Miguel Hernández-Lobato. Grammar variational autoencoder. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1945–1954, 2017.

- [14] Paweł Liskowski, Krzysztof Krawiec, Nihat Engin Toklu, and Jerry Swan. Program synthesis as latent continuous optimization: Evolutionary search in neural embeddings. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*, pages 359–367, 2020.
- [15] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic backpropagation and approximate inference in deep generative models. In *Proceedings of the 31st International Conference on Machine Learning*, volume 32 of *Proceedings of Machine Learning Research*, pages 1278–1286, 2014.
- [16] Reuven Y. Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999.
- [17] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

Appendix A

Supplementary Materials

A.1 Additional experimental results

A.2 Configuration details

A.3 Examples of optimized genotypes



© 2026 Filip Rosiak, Jakub Stefański
Poznan University of Technology
Faculty of Computing and Telecommunication
Institute of Computing Science
Typeset using L^AT_EX